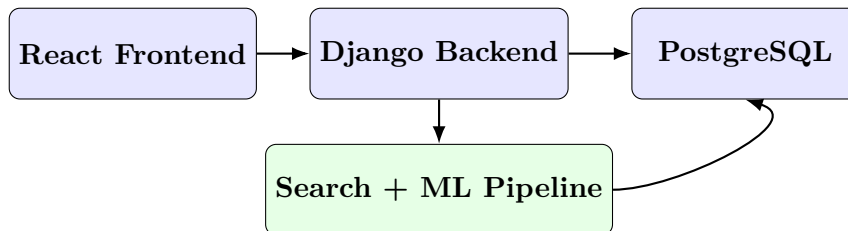


Zarailink

The Complete & Exhaustive Project Guide

A 100% deep-dive walkthrough of the Pakistani Agricultural Trade Intelligence SaaS platform — from absolute beginner to fluent contributor.



Covers: Django 4.2, DRF, PostgreSQL, React 18, Tailwind CSS, LightGBM LambdaRank, SetFit intent classifier, semantic embeddings, Node2Vec graphs, Docker, ngrok, end-to-end SaaS subscriptions.

Compiled for the Zarailink team — May 2026

Build target: `zarailink_guide.pdf`

Preface

Who is this guide for?

This document was written with one specific reader in mind: **a complete beginner who has just been handed the Zarailink codebase** and asked to make sense of it. Maybe you are a new team member joining the project. Maybe you are evaluating it as a final-year-project (FYP) supervisor. Maybe you are a student studying full-stack data products and want a worked example. In all of these cases, you should be able to read this guide top-to-bottom, with no prior knowledge of Django, React, machine learning, or the Pakistani agricultural trade domain, and emerge with a complete operational understanding of every moving part.

What "complete" means here

Most software guides paraphrase the code. This one does not. Wherever practical, we quote the *actual* source — with the file path and the exact line numbers — so you can open the file in your editor and follow along. Wherever a function uses a non-obvious algorithm (e.g. the LambdaRank loss in the Learning-to-Rank model), we explain the algorithm before we explain the code. Wherever a configuration value matters (e.g. the `SESSION_COOKIE_AGE` setting), we tell you not just what value it holds but *why that value was chosen* and what would break if it changed.

Beginner Note

You do not need to read this guide in order. Skim Chapter 1 (**Introduction**) and Chapter 3 (**Architecture**) first to get the lay of the land. Then jump to whichever chapter matches the work you are about to do.

How to use this guide

Code blocks look like this:

```
1 def hello(name: str) -> str:
2     return f"Hello, {name}!"
```

Listing 1: A code block

File references look like this: `backend/zarailink/settings.py`. When we cite a particular line, we write `backend/zarailink/settings.py:209` which means *settings.py, line 209*. You can almost always `Ctrl-G` (Go to Line) in your editor to jump straight there.

Coloured boxes flag content that needs special attention:

Beginner Note

Beginner Note — background you might be missing if you have never worked in this kind of codebase before.

Warning

Warning — a footgun. Things that have broken before, or that will break if you change them carelessly.

Tip

Tip — a shortcut, an idiom, or a way to save yourself an hour.

Worked Example

Worked Example — a concrete walkthrough of a real query, request, or workflow with sample data.

Why This Matters

Why This Matters — design rationale. Why the team chose option A over option B, and what tradeoff they accepted.

Conventions

- **Paths** are written relative to the repository root, which we write as `Zarailink-Code/`.
- **Shell commands** prefixed with `\$` are user-level; commands prefixed with `\#` require root.
- Where the project ships its own scripts, we prefer them (e.g. `./setup_linux.sh`) over manual commands so that you stay aligned with the team.
- URLs are clickable in the PDF. Any link to localhost (e.g. `http://localhost:8000`) assumes you have followed the setup chapter.

Source-of-truth disclaimer

Software changes. The exact line numbers in this guide were captured against the commit on branch `umar_branch2` (HEAD as of compilation). If a line number is off by a few when you read it, the surrounding context will still help you find the right code. The *behaviour* described, however, matches the source files committed to that branch and the corresponding ML artefacts in `backend/models/` and `backend/search/models/`.

If you spot an inaccuracy — a feature documented here that no longer exists, an endpoint that has been renamed — treat the source code as the authoritative answer and update this guide.

Contents

Preface	i
1 Introduction	1
1.1 What is Zarailink?	1
1.2 The "elevator pitch" architecture	1
1.3 The four-stage search pipeline	2
1.4 Tech stack at a glance	3
1.5 The Django apps you will encounter	3
1.6 What you can do with Zarailink (user-facing tour)	4
1.7 Roadmap of this guide	4
2 Setup — From Zero to Running	6
2.1 Hardware and OS	6
2.2 Prerequisites	6
2.2.1 Linux (Ubuntu 22/24): one-shot install	6
2.2.2 macOS (Apple Silicon or Intel)	7
2.2.3 Windows 10/11	7
2.3 Cloning the repo	8
2.4 The fast path: automated setup scripts	8
2.4.1 What the Linux script actually does	8
2.5 The slow path: manual setup	9
2.5.1 Backend	9
2.5.2 Frontend	10
2.5.3 Redis (optional but recommended)	10
2.6 The .env file (the most-edited file in your life)	10
2.6.1 Generating a SECRET_KEY	10
2.6.2 Setting up Gmail App Passwords	11
2.7 Running the application	11
2.8 Smoke test: am I really running?	12
2.9 Loading the data	12
2.10 Common setup failures	13
2.11 Demoing on a public URL with ngrok	13
3 Architecture — The Big Picture	14
3.1 The three-tier model	14
3.2 Tracing a search request end-to-end	14
3.3 Authentication and authorisation flow	16
3.4 The token-balance economy	17
3.5 Caching strategy	18
3.6 Data flow at a glance	18

3.7	The middleware stack, line by line	18
3.8	Testing pyramid (preview)	19
3.9	Where things live (the cheat sheet)	19
4	Backend Deep Dive	20
4.1	The project package: <code>zarailink/</code>	20
4.1.1	<code>settings.py</code>	20
4.1.2	<code>urls.py</code>	23
4.1.3	<code>wsgi.py</code> and <code>asgi.py</code>	23
4.2	App: <code>accounts</code>	23
4.2.1	The <code>User</code> model	23
4.2.2	The audit log (<code>accounts/audit.py</code>)	24
4.2.3	Authentication API (every endpoint)	25
4.2.4	Admin	26
4.3	App: <code>companies</code>	27
4.3.1	Models	27
4.3.2	Serializers	27
4.3.3	Endpoints (<code>companies/views.py</code>)	27
4.3.4	Admin	29
4.3.5	Management commands	29
4.4	App: <code>subscriptions</code>	30
4.4.1	Models	30
4.4.2	The paywall service: <code>subscriptions/services.py</code>	30
4.4.3	Endpoints	31
4.4.4	Admin	31
4.4.5	Management commands	31
4.5	App: <code>trade_data</code>	32
4.5.1	The product hierarchy	32
4.5.2	The <code>Transaction</code> model	32
4.5.3	Embedding tables	32
4.5.4	Ingestion commands	33
4.6	App: <code>search</code> (API surface)	33
4.6.1	Boot warmup (<code>search/apps.py</code>)	33
4.6.2	The viewset and its endpoints	34
4.6.3	Search management commands	36
4.7	Empty placeholder apps	37
4.8	The <code>utils</code> package	37
4.8.1	<code>utils/redis_client.py</code>	37
4.8.2	<code>utils/ai_service.py</code>	37
4.9	Test configuration	37
4.10	Endpoint reference summary	38
5	The Search Engine and the ML Pipeline	39
5.1	What the search engine has to do	39
5.2	The four-stage pipeline	39
5.3	Stage 1 — The NLU engine	39
5.3.1	The two parsers, side by side	39
5.3.2	What <code>ModernNLUEngine.parse</code> returns	40
5.3.3	Inside the engine, step by step	40
5.3.4	The 9-family taxonomy	42
5.3.5	NLU caching	42
5.4	Stage 2 — Subcategory resolver	42

5.4.1	The five passes	43
5.4.2	HS-code drill-down	43
5.4.3	Scope-aware availability filter	43
5.4.4	Disambiguation	43
5.5	Stage 3 — Supplier aggregator	44
5.5.1	The base aggregation	44
5.5.2	The intent-and-scope matrix	44
5.5.3	Filters	44
5.5.4	The detail and intelligence helpers	45
5.6	Stage 4 — Ranking ensemble	46
5.6.1	The 70/30 split	46
5.6.2	The eight features	46
5.6.3	The family-weight table (heuristic side)	47
5.6.4	The LightGBM LambdaRank model	47
5.6.5	The five preset rankers	48
5.7	Side quests: vector store, FAISS, OpenSearch	48
5.7.1	<code>vector_store.py</code>	48
5.7.2	<code>nlp.py</code> (legacy QueryMatcher)	48
5.7.3	OpenSearch (currently disabled)	49
5.8	Module-level singletons	49
5.9	Where the ML artefacts live	49
5.10	End-to-end trace, with code	49
5.11	The 9 known gaps (be honest with yourself)	50
6	Frontend Deep Dive	51
6.1	Tooling and configuration	51
6.1.1	<code>frontend/package.json</code>	51
6.1.2	<code>tailwind.config.js</code> and <code>postcss.config.js</code>	52
6.1.3	<code>frontend/.env</code> , <code>.env.local</code> , <code>.env.production</code>	52
6.1.4	<code>public/ assets</code>	52
6.1.5	<code>src/index.js</code> and <code>src/index.css</code>	52
6.2	The provider and route tree (<code>App.js</code>)	53
6.3	HTTP layer (<code>services/</code>)	54
6.3.1	<code>services/api.js</code>	54
6.3.2	<code>services/searchService.js</code>	55
6.4	Contexts	55
6.4.1	<code>AuthContext.js</code> (91 lines)	55
6.4.2	<code>ThemeContext.js</code> (16 lines)	55
6.5	Hooks	55
6.6	Shared components (<code>components/Common/</code>)	56
6.7	Layout	56
6.8	Marketing	57
6.9	Auth pages	57
6.10	Dashboard (<code>components/Dashboard/Dashboard.js</code> , 726 lines)	57
6.11	Search pages	58
6.11.1	<code>SearchHome.js</code> (333 lines)	58
6.11.2	<code>SearchResults.js</code> (919 lines)	58
6.11.3	<code>SummaryView.js</code> (133 lines)	59
6.11.4	<code>DataDashboard.js</code> (634 lines)	59
6.11.5	<code>ComparePage.js</code> (456 lines)	59
6.11.6	<code>DealDetail.js</code> (1,688 lines)	59
6.12	Trade Directory pages	60

6.12.1	<code>FindSuppliers.js</code> (310 lines)	60
6.12.2	<code>FindBuyers.js</code> (310 lines)	60
6.12.3	<code>CompanyProfile.js</code> (449 lines)	60
6.12.4	<code>CompanyNotes.js</code> (67 lines)	61
6.13	Trade Intelligence pages	61
6.14	Watchlist	61
6.15	Subscriptions	62
6.16	User profile	62
6.17	Cross-cutting concerns	62
6.17.1	Token economy on the frontend	62
6.17.2	Session-cookie auth model	62
6.17.3	The <code>SearchResults</code> dispatcher	62
6.18	Frontend tests	62
6.19	Open issues to be aware of	63
7	The Data Layer	64
7.1	Source spreadsheets	64
7.1.1	Inside <code>import_data_1year.xlsx</code>	64
7.1.2	Inside <code>export_data.xlsx</code>	65
7.2	JSON fixtures	65
7.3	GraphML files	65
7.4	ML model artefacts	66
7.4.1	The intent model	66
7.4.2	The search index pickle	67
7.4.3	The <code>LightGBM</code> LTR model	67
7.5	Ingestion scripts	67
7.5.1	The two <code>load_data.py</code> s	67
7.5.2	Excel ingestion commands	67
7.5.3	Companies enrichment	68
7.5.4	Search-index commands	68
7.5.5	Smart-search vector index	68
7.6	Diagnostic and dev scripts	68
7.6.1	In <code>backend/</code> root	68
7.6.2	In repo root	68
7.7	Visual assets	68
7.8	Cross-references	69
8	Deployment and Operations	70
8.1	Dockerfiles	70
8.1.1	<code>backend/Dockerfile</code>	70
8.1.2	<code>frontend/Dockerfile</code>	70
8.2	The entrypoint	71
8.3	The two docker-compose files	71
8.3.1	Root <code>docker-compose.yml</code> — <code>OpenSearch</code>	71
8.3.2	<code>backend/docker-compose.yml</code> — <code>Redis</code>	71
8.4	The three setup scripts (one per OS)	72
8.4.1	Per-OS specifics	72
8.5	Environment variables	72
8.5.1	Backend <code>.env</code>	72
8.5.2	Frontend <code>.env</code>	73
8.6	Ignore files	73
8.7	Continuous integration	73

8.8	Running in production (recommended layout)	74
8.9	Tunnelling with ngrok	74
9	Testing	76
9.1	Backend: pytest + pytest-django	76
9.1.1	Configuration	76
9.1.2	Fixtures (<code>backend/conftest.py</code>)	76
9.1.3	Test files	76
9.1.4	Running the suite	77
9.1.5	Writing a new test	77
9.2	Frontend: Jest + React Testing Library	77
9.2.1	Tests in the repo	77
9.2.2	Running	77
9.2.3	Sample frontend test	77
9.3	Ad-hoc reliability/perf scripts	78
9.4	The CI pipeline	78
9.5	Manual smoke test (the 6 checks)	78
10	Troubleshooting	79
10.1	Setup failures	79
10.2	Runtime failures	80
10.3	Frontend issues	81
10.4	Database issues	81
10.5	Performance issues	81
10.6	When all else fails	82
A	Glossary	83
B	Endpoint Reference	85
B.1	<code>/admin/...</code>	85
B.2	<code>/accounts/...</code>	85
B.3	<code>/api/</code> (companies app)	86
B.4	<code>/api/subscriptions/</code>	86
B.5	<code>/api/search/</code>	86
B.6	Catch-all	87
B.7	Auth requirements at a glance	87
C	Model Reference	88
C.1	accounts	88
C.1.1	User	88
C.1.2	AuditLog	88
C.2	companies	88
C.2.1	Sector / CompanyRole / CompanyType	88
C.2.2	Company	88
C.2.3	CompanyProduct	88
C.2.4	KeyContact	89
C.2.5	KeyContactUnlock	89
C.3	subscriptions	89
C.3.1	SubscriptionPlan	89
C.3.2	UserSubscription	89
C.3.3	TokenPurchase	89
C.3.4	RedeemCode	89

C.3.5	UserAccess	89
C.4	trade_data	89
C.4.1	HsToProductMap	89
C.4.2	Product / ProductCategory / ProductSubCategory / ProductItem	89
C.4.3	Transaction	89
C.4.4	CompanyEmbedding	90
C.4.5	ProductEmbedding	90
C.5	auditlog (third-party)	90
C.6	Models with no migrations	90
D	Quality Checklist	91
D.1	Completeness	91
D.1.1	Project introduction	91
D.1.2	Setup	91
D.1.3	Architecture	91
D.1.4	Backend	91
D.1.5	Search engine	92
D.1.6	Frontend	92
D.1.7	Data	92
D.1.8	Deployment	92
D.1.9	Testing	93
D.1.10	Troubleshooting	93
D.1.11	Reference	93
D.2	Accuracy	93
D.3	Self-consistency	93
D.4	Build hygiene	93
D.5	Pedagogy	94
D.6	Source-of-truth links	94
E	Further Resources	95
E.1	Inside the repository	95
E.1.1	Documentation	95
E.1.2	Reference assets	96
E.1.3	Schema-of-record	96
E.2	Inside this guide build	96
E.3	Open-web references	96
E.3.1	Frameworks and libraries	96
E.3.2	ML libraries	97
E.3.3	Infrastructure	97
E.3.4	Concepts	97
E.4	Where to ask for help	97
E.5	Versioning of this guide	97

List of Figures

1.1	Bird's-eye view of the Zarailink runtime topology.	2
3.1	Layered Zarailink architecture.	15
3.2	From Excel to search results: the data pipeline.	18
5.1	The four-stage search pipeline.	40

List of Tables

1.1	Zarailink technology stack.	3
1.2	Django apps and their roles.	4
2.1	Prerequisite software versions.	7
2.2	Local URLs after a successful setup.	12
3.1	One-line “where do I edit this” lookup.	19
4.1	<code>accounts.User</code> custom fields.	24
4.2	<code>accounts</code> endpoints.	25
4.3	<code>companies</code> endpoints.	28
4.4	<code>companies</code> management commands.	29
4.5	<code>subscriptions</code> endpoints.	31
4.6	Subscription plans seeded by <code>create_plans</code>	32
4.7	Highlights of the <code>Transaction</code> model.	33
4.8	<code>search</code> endpoints.	34
5.1	9-family taxonomy and triggers.	42
5.2	Intent-and-scope routing inside the aggregator.	44
5.3	The eight LTR features.	46
5.4	<code>FAMILY_WEIGHTS</code> (heuristic ranker).	47
5.5	Ranking presets.	48
5.6	Search-related ML artefacts.	49
6.1	Frontend dependencies of note.	51
6.2	Routes defined in <code>App.js</code>	54
6.3	<code>searchService.js</code> methods.	55
6.4	Shared components.	56
7.1	Source spreadsheets and CSV.	64
7.2	JSON fixtures and dumps.	65
7.3	GraphML files and their semantics.	66
7.4	ML artefacts.	66
7.5	Diagnostic scripts in <code>backend/</code> root.	69
8.1	Backend env variables.	73

Chapter 1

Introduction

1.1 What is Zarailink?

Zarailink is a Software-as-a-Service (SaaS) platform that turns millions of raw *customs trade records* from Pakistan into something a human can act on: lists of suppliers and buyers, charts of price trends, contact unlocks, and a search experience that understands natural-language queries like “*cheap rice exporters to UAE under \$0.40/kg.*”

The product targets three user personas:

- **Importers** (foreign buyers) who want to discover Pakistani suppliers of a specific commodity at a target price point.
- **Exporters** (Pakistani sellers) who want to discover overseas buyers, evaluate competitive landscapes, and price their goods.
- **Analysts and trade officers** who use the platform’s analytics, network graphs, and watchlists for market intelligence.

Beginner Note

A **customs trade record** is the line-item that a customs authority files for every shipment crossing a border. It typically includes the shipper, the consignee, the country, the HS code (a worldwide product taxonomy), the volume, and the declared price. Pakistan’s official customs data is a goldmine of business intelligence; Zarailink makes that goldmine queryable.

1.2 The "elevator pitch" architecture

At the highest level, Zarailink is a typical *three-tier web application*, but with a powerful machine-learning search engine bolted between the database and the API layer:

1. A **React** single-page application (SPA) in the browser, built with Tailwind CSS for styling and Recharts/Framer Motion for visualisations and animations.
2. A **Django + Django REST Framework (DRF)** backend that exposes REST endpoints, manages authentication, subscriptions, and the token-balance economy for unlocking key contacts.
3. A **PostgreSQL** database holding the structured trade ledger, company directory, user accounts, and subscription plans.

4. A **search-and-ranking subsystem** that sits inside the Django backend and runs a four-stage ML pipeline (NLU → retrieval → aggregation → ranking) every time the user submits a query.

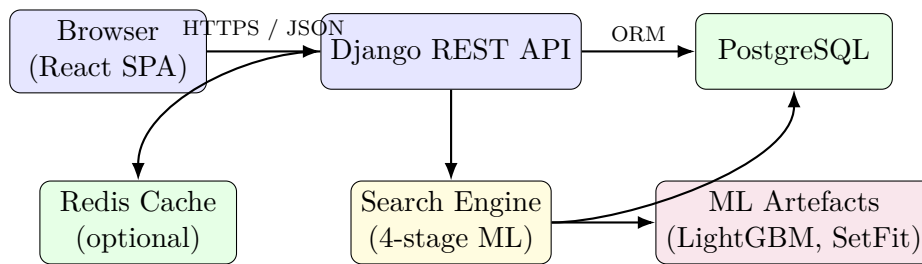


Figure 1.1: Bird's-eye view of the Zarailink runtime topology.

Why This Matters

The team chose to embed the search ML *inside* the Django process rather than running it as a separate microservice. The trade-off: simpler deployment and zero network hop, at the cost of a heavier worker process (LightGBM, SentenceTransformer, and SetFit are all loaded in memory). For a final-year project where ops simplicity matters more than horizontal scalability, this was the right call.

1.3 The four-stage search pipeline

Search is the core differentiator of Zarailink, so let us preview the pipeline now (we devote all of Chapter 5 to it).

Stage 1 — QueryInterpreter (NLU). Takes the raw query string (e.g. “*looking for cheap basmati rice from Punjab to Dubai*”) and returns a *structured intent*: an action (`BUY` / `SELL`), a product list, country filters, price operators, and so on. Implemented in `backend/search/services/nlu_engine.py` (1,305 lines).

Stage 2 — QueryMatcher. Converts the structured intent into a list of `ProductSubCategory` IDs by combining *keyword matches* (BM25), *semantic matches* (sentence embeddings) and *fuzzy matches* (RapidFuzz). Implemented in `backend/search/services/candidate_retrieval.py`.

Stage 3 — SupplierAggregator. Runs a heavy-duty Django ORM aggregation over the `Transaction` table to roll up per-supplier metrics: weighted average price, total volume, shipment frequency, repeat-customer ratio. Implemented in `backend/search/services/aggregation.py` (1,462 lines).

Stage 4 — RankingEnsemble. Ranks the suppliers via a 70/30 blend: 70% *heuristic* score (volume, recency, price-fit) and 30% *LightGBM LambdaRank* model trained on pseudo-labels. Implemented in `backend/search/services/ranking.py` and `backend/search/services/ranking_ltr.py`.

The trained LTR model lives at `backend/search/models/lgbm_ltr.txt` and the SetFit intent classifier at `backend/models/zarai_intent_model/`.

Layer	Technology	Where to find it
Frontend framework	React 18 (CRA)	<code>frontend/src/</code>
Frontend styling	Tailwind CSS, custom CSS modules	<code>frontend/tailwind.config.js</code>
Charts & animations	Recharts, Framer Motion	<code>frontend/src/components/</code>
HTTP client	Axios	<code>frontend/src/services/api.js</code>
Backend framework	Django 4.2 + DRF 3.14	<code>backend/zarailink/</code>
Auth	Session-based (<code>accounts.User</code>)	<code>backend/accounts/</code>
Subscriptions	Custom token-balance economy	<code>backend/subscriptions/</code>
Database	PostgreSQL 14+	local server, see Setup
Caching	Redis (optional) or LocMem fallback	<code>backend/zarailink/settings.py:115</code>
NLU intent	SetFit (BAAI/bge-small-en-v1.5)	<code>backend/models/zarai_intent_model/</code>
NER	GLiNER (zero-shot)	loaded in <code>nlu_engine.py</code>
Embeddings	SentenceTransformer (384-d)	loaded in <code>vector_store.py</code>
LTR ranking	LightGBM LambdaRank	<code>backend/search/models/lgbm_ltr.txt</code>
Graph features	networkx + Node2Vec	<code>backend/*.graphml</code>
Optional LLM	OpenRouter / OpenAI	<code>backend/utils/ai_service.py</code>
Container	Docker + docker-compose	<code>docker-compose.yml</code>
Tunnelling	ngrok (dev demo)	<code>backend/ngrok.exe</code>

Table 1.1: Zarailink technology stack.

1.4 Tech stack at a glance

Beginner Note

What is "session-based" auth? When you log in, Django stores a small server-side record (a "session") and sends your browser a cookie containing the session ID. On every subsequent request, the browser sends the cookie back; Django looks up the session, and knows who you are. This is the simplest auth model and what Zarailink uses despite the `django-rest-framework-simplejwt` package being listed in `requirements.txt` — the JWT plumbing is installed but not wired up.

1.5 The Django apps you will encounter

A Django project is divided into *apps*, each owning a set of models, views and URL routes. Zarailink has these:

The *empty* apps (`home`, `trade_directory`) exist as placeholders for future expansion — the team scaffolded them early but the corresponding features moved into the existing apps. This is worth noting because at first glance the app list suggests more functionality than the code actually contains; do not be misled.

App	Responsibility
accounts	Custom User model, signup, email verification, login, password reset, profile, audit log.
subscriptions	Subscription plans, redeem codes, key-contact unlocks (token economy).
companies	The company directory, sectors, key contacts, watchlist.
trade_data	The Transaction ledger and trade aggregates.
search	API surface for the search engine; orchestrates the four-stage pipeline.
home	Empty placeholder app.
trade_directory	Empty placeholder app.
trade_lens	Trade-graph / network analysis (loaded but no Django app, mostly utility code).
trade_ledger	Same: utility code, not a Django app.
market_intel	Market-intelligence helpers; mostly empty.

Table 1.2: Django apps and their roles.

1.6 What you can do with Zarailink (user-facing tour)

The frontend exposes the following pages (all routed through `frontend/src/App.js`):

- **Landing page** (`/`) — marketing copy.
- **Signup / Login / Forgot Password** (`/signup` , `/login` , `/forgot` , `/reset/<token>`).
- **Dashboard** (`/dashboard`) — charts of top commodities, top trade partners, monthly trends.
- **Search** (`/search`) — natural-language search with results, side filters, sort, pagination.
- **Deal Detail** (`/search/deal/:id`) — the massive 1,688-line component that shows everything about a single supplier-product pair: price history, partners, key contacts, verification status, watchlist toggle.
- **Compare** (`/search/compare`) — side-by-side comparison of multiple suppliers.
- **Data Dashboard** (`/search/data`) — a heavier analytical view with charts.
- **Trade Directory** (`/find-buyers` , `/find-suppliers` , `/company/:id`) — the searchable directory of companies, with a deep profile per company.
- **Trade Intelligence** (`/intelligence/...`) — company overview, partners, products, link prediction (graph ML).
- **Watchlist** (`/watchlist`) — the user’s saved companies/products.
- **Subscriptions** (`/subscription`) — plan selection, redeem code entry.
- **User Profile** (`/profile`) — account settings, token balance, audit log.

1.7 Roadmap of this guide

Chapter 1. You are here.

Chapter 2. Setting up the project on your laptop, end to end. Choose Linux, macOS, or Windows; we walk through every prerequisite, every command, every common error.

Chapter 3. The full architecture: a traced request-response, the database schema, and the pipeline contracts between modules.

Chapter 4. The Django backend: every app, every model, every endpoint.

Chapter 5. The search engine and the entire ML pipeline. NLU, retrieval, aggregation, ranking, training.

Chapter 6. The React frontend: every component, every hook, every API call.

Chapter 7. The data layer: Excel files, JSON fixtures, GraphML files, ingestion scripts, ML model artefacts.

Chapter 8. Deployment and operations: Docker, ngrok, setup scripts, environment variables.

Chapter 9. How the project is tested, plus how to add your own tests.

Chapter 10. Troubleshooting common errors.

Appendices. Glossary, API endpoint reference, model field reference, the quality checklist used to verify this guide, and external resources.

Tip

A useful first action: open `COMPLETE_SYSTEM_GUIDE.md` (in the project root) in parallel with this PDF. That markdown file is the team's internal Wiki; this PDF is the structured deep-dive that complements it.

Chapter 2

Setup — From Zero to Running

This chapter walks you all the way from a fresh laptop to a running Zarailink instance with the React UI live in your browser. We assume *nothing*: every command, every prerequisite, every gotcha is covered. If you have already done parts of this, skim — the section headings make it easy to skip.

2.1 Hardware and OS

Zarailink will run on any modern 64-bit machine. For comfort:

- **RAM:** 8 GB is the floor; 16 GB is the recommended minimum. The ML pipeline (LightGBM + SetFit + SentenceTransformer) loads several hundred MB into the Django process.
- **Disk:** reserve ~8 GB. The `backend/.venv/` directory alone is roughly 4 GB after `pip install -r requirements.txt` (PyTorch, Transformers and friends are large).
- **CPU:** any 4-core x86_64 or Apple Silicon CPU. No GPU is required for the search inference path; if you happen to have a CUDA-capable GPU, PyTorch will use it for embeddings, but the project ships fine on CPU.
- **OS:** Ubuntu 22.04+ / Debian 12+ / Fedora 39+ on Linux; macOS Sonoma or later (Intel or Apple Silicon); Windows 10/11 with WSL2 strongly recommended.

2.2 Prerequisites

You must install five pieces of system software before you touch the repo. The official requirements (mirrored from `SETUP.md`) are:

2.2.1 Linux (Ubuntu 22/24): one-shot install

```
1 # 1. System tools
2 sudo apt update
3 sudo apt install -y build-essential git curl
4
5 # 2. Python 3.12 (Ubuntu 24.04 ships with 3.12 by default)
6 sudo apt install -y python3 python3-venv python3-pip
7
8 # 3. Node.js 18 LTS via NodeSource
9 curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
10 sudo apt install -y nodejs
11
```

Tool	Min version	Why we need it
Python	3.12+	Backend runtime; <code>requirements.txt</code> pins newer-style typing.
Node.js	18 (LTS)+	Builds and serves the React frontend; the project also uses <i>Puppeteer</i> (which needs Node) for PDF export.
npm	9+	Comes with Node.js; installs frontend deps.
Docker	latest	Runs Redis Stack as a container (see <code>backend/docker-compose.yml</code>).
PostgreSQL	15+	Primary database; required even for local dev.

Table 2.1: Prerequisite software versions.

```

12 # 4. PostgreSQL 15+
13 sudo apt install -y postgresql postgresql-contrib
14 sudo systemctl enable --now postgresql
15
16 # 5. Docker
17 sudo apt install -y docker.io docker-compose-plugin
18 sudo systemctl enable --now docker
19 sudo usermod -aG docker "$USER" # log out and back in to pick this up

```

Listing 2.1: Installing all prereqs on Ubuntu

Warning

The `usermod -aG docker` change does not take effect until you **log out and back in** (or reboot). If `docker info` returns a permission error, that is why.

2.2.2 macOS (Apple Silicon or Intel)

```

1 # Homebrew itself
2 /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
3
4 # The five tools
5 brew install python@3.12 node@18 postgresql@15
6 brew services start postgresql@15
7 # Docker Desktop comes from a .dmg installer, not brew
8 # https://www.docker.com/products/docker-desktop/

```

Listing 2.2: Installing prereqs via Homebrew

2.2.3 Windows 10/11

You have two equally valid paths:

1. **Native Windows:** install Python from `python.org`, Node.js from `nodejs.org`, the PostgreSQL EDB installer, and Docker Desktop. Use the `setup_windows.ps1` script as your single source of truth.
2. **WSL2 + Ubuntu:** run the entire stack inside WSL2 and treat your machine as Linux. This is the team's recommendation because the `setup_linux.sh` script just works.

2.3 Cloning the repo

```
1 git clone <your-fork-url> Zarailink-Code
2 cd Zarailink-Code
```

Listing 2.3: Cloning Zarailink

The repository root contains `backend/`, `frontend/`, the three setup scripts, and a wealth of documentation `*.md` files.

2.4 The fast path: automated setup scripts

The Zarailink team ships three idempotent setup scripts. Idempotent means *you can run them again and again* — they skip steps that are already done. Pick the one for your OS:

OS	Command	Script
Linux	<code>./setup_linux.sh</code>	<code>setup_linux.sh</code> (395 lines)
macOS	<code>./setup_mac.sh</code>	<code>setup_mac.sh</code>
Windows	<code>.\setup_windows.ps1</code> (Power-Shell)	<code>setup_windows.ps1</code>

On Linux/macOS make the script executable first:

```
1 chmod +x setup_linux.sh # or setup_mac.sh
2 ./setup_linux.sh
```

Listing 2.4: Marking the script executable

2.4.1 What the Linux script actually does

If you want to know what is happening under the hood (and you should — this is your codebase now), here is the play-by-play of `setup_linux.sh`, broken down section by section:

Lines 66–128 — pre-flight checks. Verifies that `python3`, `python3-venv`, `node`, `npm`, `docker`, and `psql` all exist. Bails out with a friendly install hint if any are missing.

Lines 138–157 — virtualenv. Creates `backend/.venv/` via `python3 -m venv .venv`. If a `venv` exists but is broken (missing `bin/activate`), it deletes and recreates.

Lines 161–191 — pip install. Activates the `venv` and runs `pip install -r requirements.txt`. This installs ~120 packages including Django, DRF, LightGBM, SentenceTransformers, SetFit, GLiNER, KeyBERT, OpenSearch, and more. *Allow up to 5 minutes here.*

Lines 194–207 — .env file. Copies `backend/.env.example` to `backend/.env` and generates a fresh `SECRET_KEY` via Django’s helper.

Lines 215–228 — PostgreSQL database. Calls `sudo -u postgres createdb zarailink` and sets the `postgres` user’s password to `postgres` (yes, really — it is hardcoded in `settings.py:105`).

Lines 231–237 — migrations and roles. Runs `python manage.py migrate` to create all tables, then `python manage.py setup_company_roles` to seed the company-role lookup data.

Lines 244–251 — sample company data. If `backend/load_data.py` exists and the `Company` table is empty, loads sample data.

Lines 257–266 — trade transactions. If `import_data_1year.xlsx` exists at the repo root, ingests it via `python manage.py ingest_trade -file ...`. This populates the `Transaction` table that the search engine and dashboards depend on.

Lines 269–297 — GNN graphs and embeddings. If transactions exist, builds the `*.graphml` files via `build_gnn_graphs` and the `CompanyEmbedding` rows via `generate_gnn_embeddings -fast`. These power the “Similar Companies” feature.

Lines 305–320 — Redis container. Starts `backend/docker-compose.yml` which launches a `redis/redis-stack:latest` container named `zarailink-redis` on port `6379` (data) and `8001` (Redis Insight UI).

Lines 327–336 — frontend npm install. `cd ../frontend && npm install -legacy-peer-deps` installs React, Tailwind, Recharts, etc.

Lines 343–351 — Puppeteer. In `backend/`, a tiny `package.json` pulls in Puppeteer for server-side PDF generation (used by the report-export endpoints).

Lines 358–393 — summary banner. Prints the next-step commands and access URLs.

Tip

If the script bails out part-way through, you can rerun it. It will pick up where it left off thanks to the idempotency checks. The most common failure is the `pip install` step running out of memory on small VMs — swap or upgrade to 8 GB+ RAM.

2.5 The slow path: manual setup

If the automated script does not work for you (corporate firewall, custom OS, etc.), here is the equivalent step-by-step you can run by hand.

2.5.1 Backend

```

1 cd backend
2
3 # 1. Virtual environment
4 python3 -m venv .venv
5 source .venv/bin/activate          # on Windows: .\.venv\Scripts\Activate.ps1
6
7 # 2. Dependencies
8 pip install --upgrade pip
9 pip install -r requirements.txt
10
11 # 3. Environment file
12 cp .env.example .env
13
14 # 4. Generate a SECRET_KEY and paste it into .env
15 python -c "from django.core.management.utils import get_random_secret_key; \
    print(get_random_secret_key())"
16
17 # 5. Database
18 sudo -u postgres createdb zarailink    # Linux
19 # brew: createdb zarailink            # macOS

```

```

20 # Windows: createdb -U postgres zarailink
21
22 # 6. Apply migrations
23 python manage.py migrate
24
25 # 7. Optional: superuser for /admin
26 python manage.py createsuperuser
27
28 # 8. Seed lookup tables
29 python manage.py setup_company_roles
30
31 # 9. Optional: load sample companies
32 python load_data.py
33
34 # 10. Run the dev server
35 python manage.py runserver 0.0.0.0:8000

```

Listing 2.5: Manual backend setup

2.5.2 Frontend

```

1 cd frontend
2 npm install --legacy-peer-deps
3 npm start

```

Listing 2.6: Manual frontend setup

2.5.3 Redis (optional but recommended)

```

1 cd backend
2 docker compose up -d          # newer Docker
3 # or
4 docker-compose up -d          # older Docker

```

Listing 2.7: Starting Redis via Docker

Beginner Note

Redis is **optional**. `backend/zarailink/settings.py:115-143` contains an `is_redis_available()` check that opens a TCP socket to `127.0.0.1:6379`. If Redis is up, Django uses `django_redis.cache.RedisCache`. If it is down, Django silently falls back to in-process `LocMemCache`. So you can develop without Docker; you just lose cross-process cache sharing and the NLU cache benefit.

2.6 The .env file (the most-edited file in your life)

The setup script copies `backend/.env.example` to `backend/.env`. Here is the full list of variables and what they do, cross-checked against `backend/zarailink/settings.py`:

2.6.1 Generating a SECRET_KEY

Variable	Required?	Effect
SECRET_KEY	Yes	Django cryptographic signing key. Read at <code>settings.py:28</code> .
OPENROUTER_API_KEY	No	Used by <code>utils/ai_service.py</code> for the LLM-backed price extraction fallback. <code>settings.py:30</code> .
OPENAI_KEY	No	Used for “AI Smart Search” and the OpenAI embedding fallback. <code>settings.py:146</code> .
EMAIL_HOST_USER	No	Gmail address for outbound email (verification, password reset). <code>settings.py:327</code> .
EMAIL_HOST_PASSWORD	No	Gmail App Password , not your regular password.
FRONTEND_URL	No	Used inside email templates so that “Verify your account” links point to the correct host. Defaults to <code>http://localhost:3000</code> . <code>settings.py:332</code> .

```

1 cd backend && source .venv/bin/activate
2 python -c "from django.core.management.utils import get_random_secret_key; ↵
    print(get_random_secret_key())"

```

Listing 2.8: One-liner for SECRET_KEY

Copy the output into `backend/.env` as `SECRET_KEY=django-insecure-xxxxxxxxxxxxxxxxxxxxxxxxxxxx`.

2.6.2 Setting up Gmail App Passwords

If you want email verification to work end-to-end, you need a Gmail App Password (the kind that is immune to 2FA prompts):

1. Visit <https://myaccount.google.com/security> and turn on *2-Step Verification* (cannot generate App Passwords without it).
2. Visit <https://myaccount.google.com/apppasswords>.
3. Create a new App Password called “Zarailink”. Copy the 16 characters Google gives you.
4. Paste it into `.env` as `EMAIL_HOST_PASSWORD`, with no spaces.

Warning

Do **not** commit your `.env` file. It is already listed in `.gitignore`. If you accidentally commit it, rotate every secret inside.

2.7 Running the application

Three terminals, three commands:

```

1 cd backend
2 docker compose up -d

```

Listing 2.9: Terminal 1: Redis (optional)

```

1 cd backend

```

```

2 source .venv/bin/activate
3 python manage.py runserver 0.0.0.0:8000

```

Listing 2.10: Terminal 2: Django backend

```

1 cd frontend
2 npm start

```

Listing 2.11: Terminal 3: React frontend

After a few seconds you can open the URLs in Table 2.2.

Service	URL
Frontend (React)	http://localhost:3000
Backend API (DRF)	http://localhost:8000/api/
Django admin	http://localhost:8000/admin/
Redis Insight UI	http://localhost:8001
DRF browsable login	http://localhost:8000/api-auth/login/
CKEditor 5 endpoint	http://localhost:8000/ckeditor5/

Table 2.2: Local URLs after a successful setup.

2.8 Smoke test: am I really running?

Six checks — if all six pass, you have a working install.

1. `curl http://localhost:8000/admin/` returns HTML containing the Django log-in page.
2. `curl http://localhost:8000/api/` returns a JSON 401 (auth required) or 200 with route hints — both are fine.
3. Open <http://localhost:3000> — you see the Zarailink landing page styled with Tailwind.
4. Sign up at <http://localhost:3000/signup>; you receive a verify email if SMTP is configured (or you skip verification by setting `is_verified=True` in the admin).
5. Visit <http://localhost:3000/dashboard> after login: you see charts (they may be empty if you skipped data ingestion).
6. Type a query in the search box at <http://localhost:3000/search> — you receive results within ~2 seconds.

2.9 Loading the data

A search engine without data is just a spinning wheel. The repo ships sample fixtures and Excel files which you can ingest:

```

1 cd backend && source .venv/bin/activate
2
3 # Companies + sectors + roles
4 python manage.py setup_company_roles
5 python load_data.py
6
7 # Trade transactions (import side)
8 python manage.py ingest_trade --file ../import_data_1year.xlsx
9
10 # Trade transactions (export side)
11 python manage.py ingest_trade_export --file ../export_data.xlsx
12

```

```

13 # Build the search index pickle
14 python manage.py build_search_index
15
16 # Optional: GNN graphs and embeddings (enables Similar Companies)
17 python manage.py build_gnn_graphs
18 python manage.py generate_gnn_embeddings --fast

```

Listing 2.12: Loading sample data

We dedicate Chapter 7 to a deep dive on these data files, their schemas, and the ingestion pipeline.

2.10 Common setup failures

We collect the recurring failure modes here so you can find them by `Ctrl-F` and move on.

`ModuleNotFoundError: No module named 'django'`. You forgot `source .venv/bin/activate`.

`FATAL: database "zarailink" does not exist`. Run `sudo -u postgres createdb zarailink` (Linux) or `createdb zarailink` (macOS).

`FATAL: password authentication failed for user "postgres"`. Run `sudo -u postgres psql -c "ALTER USER postgres PASSWORD 'password'"`. Yes, the password is hardcoded.

Redis connection refused. Either start the container (`docker compose up -d` from `backend/`) or ignore — Django will fall back to `LocMemCache`.

`npm ERR! ERESOLVE unable to resolve dependency tree`. Re-run with `npm install --legacy-peer-deps`.

`pip install stalls on torch`. Torch wheels are large (~700 MB). Be patient, or pre-install `torch==2.1.0` from the PyTorch index.

`psql: command not found`. Add the PostgreSQL `bin` folder to your `PATH`.

Browser shows CSRF verification failed. Make sure `CORS_ALLOW_CREDENTIALS=True` (it is, in `settings.py:349`) and that the frontend dev server is on `localhost:3000`, not `127.0.0.1:3000`.

2.11 Demoing on a public URL with ngrok

The repo ships `backend/ngrok.exe` (Windows) so the team can quickly expose `localhost:8000` during demos. The settings file already whitelists ngrok hosts at `settings.py:36`:

```

36 ALLOWED_HOSTS = ['localhost', '127.0.0.1', '.ngrok-free.dev', '.ngrok-free.app↵
    ' ]

```

Listing 2.13: ngrok-friendly ALLOWED_HOSTS

To run a demo tunnel:

```

1 ngrok http 8000 # gives you a https://xxx.ngrok-free.app URL

```

Listing 2.14: Public ngrok tunnel

You should now have a fully running Zarailink stack on your machine. The next chapter zooms out and explains *how* all the pieces talk to each other.

Chapter 3

Architecture — The Big Picture

Now that you can run Zarailink, this chapter explains *how it works*. By the end you will be able to follow a request from the moment a user clicks *Search* all the way to the JSON response, and you will know the role of every layer in between.

3.1 The three-tier model

Zarailink follows the classic web-app trichotomy:

1. **Presentation** — the React SPA in the browser. Renders UI, captures input, makes HTTP calls.
2. **Application** — the Django + DRF process. Authenticates, authorises, runs business logic, talks to ML, and queries the DB.
3. **Persistence** — PostgreSQL (durable) plus optional Redis (cache).

The novelty is the **search-and-ranking subsystem** that lives inside the application tier (Figure 3.1). The team chose to embed it in the Django process for ops simplicity: no extra service to start, no extra port to firewall, no separate deploy. The cost is memory: at boot, the worker loads the LightGBM model, the SetFit transformer, and a SentenceTransformer encoder.

3.2 Tracing a search request end-to-end

This is the single best way to internalise the architecture: walk through what happens when a user submits a search query.

Worked Example

Scenario. A logged-in user types “*cheap basmati rice from Punjab to UAE under \$0.40/kg*” into the search bar and presses Enter.

1. **[Browser]** `frontend/src/components/Search/SearchHome.js` captures the keystroke, calls `searchService.search(query)`.
2. **[Browser → Network]** Axios issues `POST http://localhost:8000/api/search/` with body `\{"q": "cheap basmati ..."\}` and the session cookie attached (`withCredentials: true` from `frontend/src/services/api.js`).

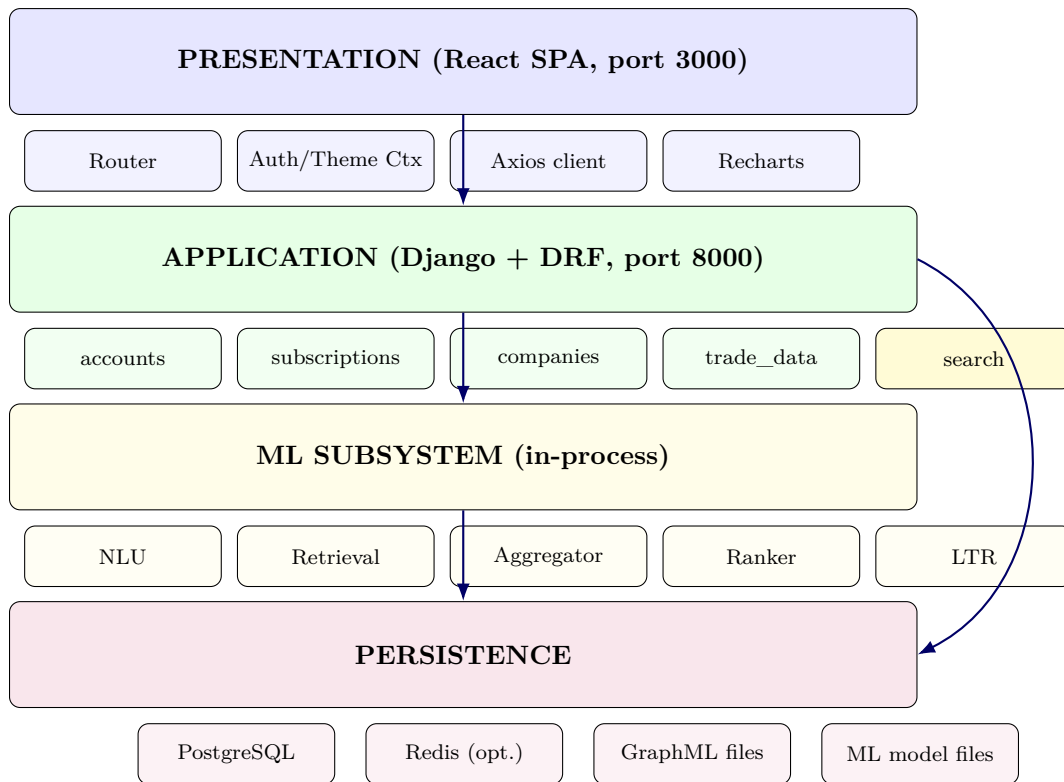


Figure 3.1: Layered Zarailink architecture.

3. **[Django middleware]** The request flows through the middleware stack listed in `settings.py:65-75` : Security → Session → CORS → Common → CSRF → Auth → Messages → Clickjacking → Auditlog. The user object gets attached to `request.user` during the Auth step.
4. **[URL resolver]** `backend/zarailink/urls.py:33` forwards `api/search/` to `backend/search/urls.py`.
5. **[ViewSet]** The matching DRF action in `backend/search/views.py` runs.
6. **[SearchService]** The view delegates to `backend/search/services/search\_service.py:SearchService`. This is the orchestrator.
7. **[Stage 1 — NLU]** `ModernNLUEngine.parse(query)` runs in `nl_engine.py` :
 - (a) **Cache check** (Redis or LocMem). MD5 hash of the query; on hit, return cached structured intent.
 - (b) **Intent classification** via SetFit (`BUY` vs `SELL`) using the model in `backend/models/zarai_intent_mo`
 - (c) **Entity extraction** via GLiNER — pulls country names, product mentions, quantities, price operators in one pass.
 - (d) **Keyword isolation** via KeyBERT — strips “urgently looking for”, “please send quotation”, etc.
 - (e) **Geographic resolution** via RapidFuzz — normalises “Pak” → “Pakistan”, “UAE” → “United Arab Emirates”.
 - (f) **Price logic** via regex first, falling back to OpenRouter LLM only if regex extraction fails. The result is a structured dict like:

```
{
  "action": "BUY",
  "products": ["basmati rice"],
  "from_country": "Pakistan",
```

```

    "to_country": "United Arab Emirates",
    "price_op": "<=",
    "price_value": 0.40,
    "price_currency": "USD",
    "price_unit": "kg",
    "raw_query": "cheap basmati rice from Punjab to UAE under $0.40/kg"
  }

```

Listing 3.1: Example NLU output

8. **[Stage 2 — Candidate retrieval]** `backend/search/services/candidate\_retrieval.py` maps the product list to a set of `ProductSubCategory` IDs by:
 - Loading `backend/search\_index.pkl` (in-memory).
 - Running BM25 over product names.
 - Running cosine similarity against the SentenceTransformer embeddings (`vector_store.py`).
 - Running RapidFuzz fuzzy match for typos.
 - Merging the three rankings.
9. **[Stage 3 — Aggregation]** `backend/search/services/aggregation.py:SupplierAggregator` runs a single Postgres query (the heaviest in the pipeline) that:
 - Filters `Transaction` rows by product subcategory, from-country, to-country, and price range.
 - Groups by supplier company.
 - Computes `SUM(volume)` , `AVG(price)` (volume-weighted), `COUNT` (shipment frequency), `MAX(date)` (recency), and buyer-concentration metrics.
10. **[Stage 4 — Ranking]** `backend/search/services/ranking\_ltr.py:RankingEnsemble` scores each candidate. The ensemble is:

$$\text{score}_i = 0.7 \text{ heuristic}(i) + 0.3 \text{ LightGBM}(i)$$

The heuristic side uses min-max-scaled volume, recency, frequency, price-fit; the LightGBM side runs the LambdaRank model from `backend/search/models/lgbm\_ltr.txt` .
11. **[Response]** The view serialises a list of supplier dicts and returns JSON over HTTPS.
12. **[Browser]** `SearchResults.js` renders the list, wires up filters, sort, pagination, and clicking a result navigates to `DealDetail.js` .

Why This Matters

Why a four-stage pipeline? Each stage has a different cost model: NLU is cheap and cacheable, retrieval is moderately heavy but bounded, aggregation hits PostgreSQL, ranking is in-memory math. Splitting the pipeline lets the team *cache aggressively at the cheap end* and parallelise / scale the expensive end independently. It also lets unit-tests target one stage at a time — a virtue you will appreciate when debugging a search miss.

3.3 Authentication and authorisation flow

Zarailink uses Django's built-in session-cookie auth. The flow:

1. **Signup** (`POST /accounts/api/signup/`). Body contains `name`, `email`, `password`, `country`. The view constructs a `UserRegisterForm`, saves the user, marks `is_active = True` and `email_verified = True` immediately (the form's `is_active=False` is overridden).
2. **Login** (`POST /accounts/api/login/`). The view calls Django's `authenticate(...)` then `login(request, user)` which writes a session cookie named `sessionid`.
3. **Subsequent requests**. The browser includes the `sessionid` cookie (axios is configured with `withCredentials: true`). On the server, `SessionMiddleware` loads the session, then `AuthenticationMiddleware` attaches `request.user`.
4. **Permission check**. Each protected view starts with `if not request.user.is_authenticated: return 401`. DRF `ViewSet`s use `permission_classes` on the class.
5. **Logout** (`POST /accounts/api/logout/`). The view calls `logout(request)` which destroys the session.

Beginner Note

Why session-based and not JWT? JWTs are stateless — the server needs no DB lookup to validate them. That is great for horizontally scaled, polyglot back ends. Sessions, by contrast, are simpler to revoke (just delete the row) and avoid the perennial JWT footguns (expiry, refresh tokens, key rotation). For a Django monolith with a single backend, the session model is plainly easier. The presence of `django-rest-framework-simplejwt` in `requirements.txt` is a leftover; nothing in the codebase wires it up.

3.4 The token-balance economy

A core product mechanic: every user has a `token_balance` (integer, default 0; `accounts/models.py:48`). To unlock a **key contact's** phone or email, the user spends one token. The ledger row is a `KeyContactUnlock` (`companies/models.py:167`).

The token economy is sustained by:

- **Subscription plans** (`subscriptions/models.py:SubscriptionPlan`) — monthly token grants.
- **Redeem codes** (`subscriptions/models.py:RedeemCode`) — one-shot bundles purchased from the team.
- **Token purchases** (`subscriptions/models.py:TokenPurchase`) — direct top-ups.
- **Admin grants** — Django admin actions (`accounts/admin.py:add_100_tokens`, `add_1000_tokens`) for support cases.

The authority for “does this user have access?” is in `backend/subscriptions/services.py:get_access_state` and `purchase_access`. A view that wants to gate a feature behind tokens calls these helpers; they encapsulate the “check balance, deduct, record audit log” transaction in one place so feature views stay clean.

3.5 Caching strategy

- **NLU cache.** The structured intent dict for a query is keyed by MD5 hash of the lowercased, stripped query. TTL: 24 hours. On a Redis-backed deploy this is shared across worker processes; on LocMem it is per-process. See `nlu_engine.py` (cache hooks).
- **Search-index pickle.** The product subcategory $\rightarrow$ id lookup, BM25 index, and embedding matrix are all serialised into a single `backend/search\_index.pkl` loaded once at boot.
- **Django ORM ‘select_related’ / ‘prefetch_related’.** Hot endpoints (e.g. `search/views.py:supplier\`) batch the foreign-key joins to avoid N+1 queries.
- **Redis Insight UI.** The team can inspect cache contents live at <http://localhost:8001> when the `redis-stack` container is running.

3.6 Data flow at a glance

Figure 3.2 traces the data lineage from the original Excel files all the way to a search result.

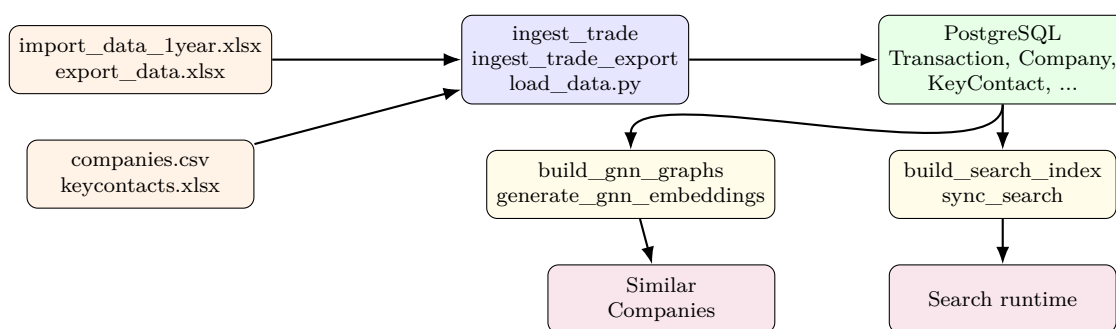


Figure 3.2: From Excel to search results: the data pipeline.

3.7 The middleware stack, line by line

The order of `settings.py:65-75` matters; here is what each layer does and why it is at that position:

1. **SecurityMiddleware** — HTTPS redirects, HSTS headers, XSS filter. Must be early so it can short-circuit insecure requests.
2. **SessionMiddleware** — loads the session from the `sessionid` cookie. Must precede `AuthenticationMiddleware`.
3. **CorsMiddleware** — adds CORS headers. Django’s docs require it before `CommonMiddleware` so that CORS responses are emitted even when Common slash-redirects.
4. **CommonMiddleware** — URL normalisation, ETag handling.
5. **CsrfViewMiddleware** — enforces CSRF tokens on unsafe methods. Must run before view dispatch.
6. **AuthenticationMiddleware** — attaches `request.user`.
7. **MessageMiddleware** — the Django flash-message system.
8. **XFrameOptionsMiddleware** — sets `X-Frame-Options: DENY`.

9. **AuditlogMiddleware** — pulls the user from the request and stores it on a thread-local so model-save signals can record “who made this change”.

3.8 Testing pyramid (preview)

The repo ships an extensive `conftest.py` with 28 reusable fixtures (users, companies, plans, redeem codes, contacts, etc.) plus `pytest.ini`. Tests live in:

- `backend/accounts/tests/` — model + view tests.
- `backend/companies/tests/test_views.py`.
- `backend/subscriptions/tests/test_views.py`.
- `backend/search/tests/test_scope_consistency.py`.
- Several loose `test_*.py` files in `backend/` root used as ad-hoc reliability harnesses (`reliability_test.py`, `perf_test.py`, `diagnose_typo_queries.py`, etc.).

We dedicate Chapter 9 to the test suite.

3.9 Where things live (the cheat sheet)

If you want to change...	Open...
A backend URL	<code>backend/<app>/urls.py</code>
A backend endpoint's behaviour	<code>backend/<app>/views.py</code>
A database field	<code>backend/<app>/models.py</code> (then <code>makemigrations</code>)
The search ranking weights	<code>backend/search/services/ranking.py</code>
The NLU intent labels	<code>backend/search/services/nlu_engine.py</code>
A search-result React component	<code>frontend/src/components/Search/</code>
The auth context state	<code>frontend/src/context/AuthContext.js</code>
Tailwind theme	<code>frontend/tailwind.config.js</code>
Subscription plans	<code>backend/subscriptions/management/commands/create_plans.py</code>

Table 3.1: One-line “where do I edit this” lookup.

Chapter 4

Backend Deep Dive

This chapter dissects every Django app and every endpoint in the backend. We assume you have skim-read Chapter 3; here we zoom in.

The backend is rooted at `backend/` and contains the project package `backend/zarailink/` plus seven application packages. We cover each in order.

4.1 The project package: `zarailink/`

Five files: `__init__.py` (empty), `settings.py` (392 lines), `urls.py` (37 lines), `wsgi.py` and `asgi.py`.

4.1.1 `settings.py`

This is the single source of truth for runtime configuration.

Path & env loading (lines 13–21):

```
13 from pathlib import Path
14 import os
15 from dotenv import load_dotenv
16 BASE_DIR = Path(__file__).resolve().parent.parent # /backend
17 load_dotenv(BASE_DIR / '.env')
```

`BASE_DIR` resolves to `backend/` because `settings.py` sits one level deep inside `zarailink/` and `.parent.parent` walks up twice. The `load_dotenv` call happens before any setting is read, so anything in `.env` becomes available via `os.getenv`.

Secrets (lines 28–30, 146):

```
SECRET_KEY          = os.getenv('SECRET_KEY', '')
OPENROUTER_API_KEY  = os.getenv('OPENROUTER_API_KEY', '')
OPENAI_API_KEY       = os.getenv('OPENAI_KEY', '') # NOTE: the env var is ↩
                    OPENAI_KEY
```

Warning

`OPENAI_KEY` (no `API`) is the env var, but the Python attribute is `OPENAI_API_KEY`. Mismatching these is a common bug.

Debug & hosts (lines 34–36):

```
DEBUG = True
ALLOWED_HOSTS = ['localhost', '127.0.0.1', '.ngrok-free.dev', '.ngrok-free.app↵
']
```

`DEBUG = True` is hard-coded; for production you must change this manually.

INSTALLED_APPS (lines 41–63):

```
41 INSTALLED_APPS = [
42     'django.contrib.admin',
43     'django.contrib.auth',
44     'django.contrib.contenttypes',
45     'django.contrib.sessions',
46     'django.contrib.messages',
47     'django.contrib.staticfiles',
48     'rest_framework',
49     'corsheaders',
50     'django_ckeditor_5',
51     'auditlog',
52     'accounts',
53     'subscriptions',
54     'companies',
55     'trade_data',
56     'search',
57     'django.contrib.postgres',
58     'django_extensions',
59 ]
```

Notably absent: the empty `home` and `trade_directory` apps. Their directories exist but they are not installed.

Middleware order (lines 65–75):

```
65 MIDDLEWARE = [
66     'django.middleware.security.SecurityMiddleware',
67     'django.contrib.sessions.middleware.SessionMiddleware',
68     'corsheaders.middleware.CorsMiddleware',
69     'django.middleware.common.CommonMiddleware',
70     'django.middleware.csrf.CsrfViewMiddleware',
71     'django.contrib.auth.middleware.AuthenticationMiddleware',
72     'django.contrib.messages.middleware.MessageMiddleware',
73     'django.middleware.clickjacking.XFrameOptionsMiddleware',
74     'auditlog.middleware.AuditlogMiddleware',
75 ]
```

Templates (lines 79–92): the project searches three template roots: `backend/templates/`, `backend/accounts/templates/`, and `frontend/build/` (the React production build's `index.html`).

The catch-all URL at the end of `urls.py:36` renders that file for any non-API URL — this is how the SPA gets shipped.

Database (lines 100–109):

```
100 DATABASES = {
101     'default': {
102         'ENGINE': 'django.db.backends.postgresql',
103         'NAME': 'zarailink',
104         'USER': 'postgres',
105         'PASSWORD': 'postgres',
106         'HOST': 'localhost',
107         'PORT': '5432',
108     }
109 }
```

Redis auto-detect (lines 113–143). The pattern is:

```
115 def is_redis_available():
116     try:
117         sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
118         sock.settimeout(1)
119         result = sock.connect_ex(('127.0.0.1', 6379))
120         sock.close()
121         return result == 0
122     except:
123         return False
```

If the TCP probe succeeds, Django uses `django_redis.cache.RedisCache` on `redis://127.0.0.1:6379/0`; otherwise it falls back to `django.core.cache.backends.locmem.LocMemCache`.

Logging (lines 149–187): two formatters (`verbose`, `simple`), two handlers (`console` at `DEBUG`, `file` at `ERROR` writing to `backend/debug.log`), two named loggers (`django` at `INFO`, `zarailink` at `DEBUG`).

Auth wiring :

- `settings.py:209` `AUTH_USER_MODEL = 'accounts.User'`.
- `settings.py:212` `LOGIN_REDIRECT_URL = '/dashboard'`.
- `settings.py:213` `LOGOUT_REDIRECT_URL = '/login'`.

CORS (lines 336–369): `localhost` on ports `3000/3001/4000` are explicitly whitelisted, plus a regex allowing any `http://localhost:<port>`. `CORS_ALLOW_CREDENTIALS = True` is essential for cookie-based auth to flow cross-origin.

Session security (lines 372–391):

```
372 CSRF_TRUSTED_ORIGINS = [...same localhost list...]
373 CSRF_COOKIE_HTTPONLY = False # JS must read it for X-CSRFToken header
374 CSRF_COOKIE_SAMESITE = 'Lax'
375 SESSION_COOKIE_SAMESITE = 'Lax'
```

```

376 SESSION_COOKIE_HTTPONLY = True
377 SESSION_COOKIE_SECURE    = False # dev only
378 SECURE_CONTENT_TYPE_NOSNIFF = True
379 SECURE_BROWSER_XSS_FILTER = True
380 X_FRAME_OPTIONS           = 'DENY'
381 SESSION_COOKIE_AGE        = 1209600 # 14 days
382 SESSION_SAVE_EVERY_REQUEST = True
383 SESSION_EXPIRE_AT_BROWSER_CLOSE = False

```

4.1.2 urls.py

The URL root mounts each app at a stable prefix:

```

26 urlpatterns = [
27     path('favicon.ico', favicon),
28     path('admin/', admin.site.urls),
29     path('accounts/', include('accounts.urls')),
30     path('api/subscriptions/', include('subscriptions.urls')),
31     path('api/', include('companies.urls')), # mounted at /api/, not /api/↔
        companies/
32     path('api/search/', include('search.urls')),
33     path("ckeditor5/", include('django_ckeditor_5.urls')),
34     path('api-auth/', include('rest_framework.urls')),
35     re_path(r'^.*$', TemplateView.as_view(template_name='index.html')),
36 ]

```

Warning

The `re_path(r'\^.*$', ...)` catch-all is the React SPA fallback. It must remain *last*, otherwise it will swallow API routes.

4.1.3 wsgi.py and asgi.py

Both are stock Django bootstraps that set `DJANGO_SETTINGS_MODULE = 'zarailink.settings'` and call `get_wsgi_application()` or `get_asgi_application()` respectively. Use `wsgi.py` with `unicorn/uWSGI`; use `asgi.py` with `Daphne/Uvicorn` for async.

4.2 App: accounts

The custom user model and the entire authentication API. App config: `AccountsConfig` with `default_auto_field = BigAutoField`.

4.2.1 The User model

`backend/accounts/models.py` defines exactly two classes: `CustomUserManager` (lines 9–35) and `User` (lines 38–105).

The manager swaps Django's `username` for `email` as the unique identifier:

```

14 def create_user(self, email, password=None, username=None, **extra_fields):
15     if not email:
16         raise ValueError(_('The Email must be set'))

```

```

17     email = self.normalize_email(email)
18     user = self.model(email=email, username=username, **extra_fields)
19     user.set_password(password)
20     user.save(using=self._db)
21     return user

```

The `User` model adds these custom fields on top of `AbstractUser` :

Field	Type	Options / purpose
email	EmailField	unique=True (used as login)
username	CharField(150)	nullable; demoted to optional
bio	CharField(500)	default empty string
country	CharField(100)	nullable
email_verified	BooleanField	default False
verification_token	UUIDField	default=uuid.uuid4, unique=True
token_created_at	DateTimeField	default timezone.now
phone_number	CharField(30)	nullable
job_title	CharField(100)	nullable
token_balance	IntegerField	default 0 (the token economy)

Table 4.1: `accounts.User` custom fields.

`USERNAME_FIELD = "email"` (line 63) plus `REQUIRED_FIELDS = ["first_name", "last_name"]` make the model log-in via email and require first/last name on `createsuperuser` .

Methods worth memorising :

- `is_verification_token_valid()` — returns `False` if already verified, otherwise compares `token_created_at + 24h` to `now()` . The 24-hour expiry on email verification lives here.
- `regenerate_verification_token()` — assigns a fresh UUID and resets `token_created_at` (used by the “resend verification email” endpoint).
- `has_tokens(amount=1)` — boolean balance check.
- `deduct_tokens(amount=1)` — atomic deduct, returns success bool.
- `add_tokens(amount)` — credit. Returns `None` (no boolean).

4.2.2 The audit log (`accounts/audit.py`)

`backend/accounts/audit.py` defines an `AuditLog` model. **Caveat:** Django auto-discovers models inside `models.py` ; placing them in `audit.py` means no migration is auto-generated. The file is currently *infrastructure-in-waiting*: the helpers `log_action(...)` and `get_user_activity(...)` are imported by views but the table is not created without a manual migration.

The third-party `auditlog` app (registered in `INSTALLED_APPS`, configured at the bottom of `companies/models.py`) is the live audit trail.

4.2.3 Authentication API (every endpoint)

`backend/accounts/views.py` contains 9 function-based views and one HTML view. Mounted under `accounts/` via the project URLs. Below is the complete table, then a per-endpoint walkthrough.

M.	URL	Purpose
GET/P	<code>accounts/register/</code>	HTML form (Django template)
GET/P	<code>accounts/login/</code>	Same view, alias
POST	<code>accounts/api/signup/</code>	JSON signup
POST	<code>accounts/api/login/</code>	JSON login (sets session cookie)
POST	<code>accounts/api/logout/</code>	Destroys session
GET	<code>accounts/api/check-auth/</code>	Bootstrap session for SPA
POST	<code>accounts/api/forgot-password/</code>	Sends reset email
GET	<code>accounts/api/verify-email/<uuid:token>/</code>	Verifies, redirects to React
POST	<code>accounts/api/resent-verification/</code>	Re-sends multi-part HTML+text email
POST	<code>accounts/api/update-profile/</code>	Auth-required
POST	<code>accounts/api/change-password/</code>	Auth-required, re-auths session
GET	<code>accounts/password_reset_confirm/<uidb64:token>/</code>	Django's stock view
GET	<code>accounts/password_reset_complete/</code>	Django's stock view

Table 4.2: `accounts` endpoints.

POST `/accounts/api/signup/` (lines 38–76). Body: `\{name, email, password, country\}`. Internally constructs a `UserRegisterForm`, but *overrides* the form's `is_active=False` by setting both `is_active = True` and `email_verified = True` immediately.

Why This Matters

The form was originally written for an email-verification flow; the team short-circuited it for the demo so users can sign up and use the product right away. If you ever turn email verification back on, remove the `user.email_verified = True` line and re-enable `is_active = False`.

POST `/accounts/api/login/` (lines 79–105). Calls `authenticate(request, email=..., password=...)` (which works because `USERNAME_FIELD = 'email'`) then `login(request, user)`. The session cookie is set by Django automatically. The 200 response includes `\{success, user: \{name, email, email_veri`

POST `/accounts/api/logout/` (lines 108–111). Calls `logout(request)` and returns `\{"success": true\}`. No HTTP-method check.

GET `/accounts/api/check-auth/` (lines 114–129). The SPA calls this on page load. If `request.user.is_authenticated`, returns the same user payload as login; else `\{"authenticated": false\}`.

POST `/accounts/api/forgot-password/` (lines 132–157). Body: `\{email\}`. Always returns `\{"success": true\}` regardless of whether the email is registered (anti-enumeration). On success, sends an email containing the Django default token + base64-encoded UID, pointing back at `accounts:password_reset_confirm`.

GET `/accounts/api/verify-email/<uuid:token>/` (lines 160–190). Looks up the user by token. Branches:

- Already verified → redirects to `\{FRONTEND_URL\}/verify-email/\{token\}?status=already_verified`.
- Token expired (> 24h) → redirects with `?status=expired`.
- Token valid → marks verified & active, redirects with `?status=verified`.
- Token not found → redirects with `?status=invalid`.

POST `/accounts/api/resend-verification/` (lines 193–313). Body: `\{email\}`. Re-rolls the verification token, then composes a *multi-part* email with both an HTML body (branded green `\#1A4D2E`, “Optimize Data. Empower Tomorrow.” tagline) and a plain-text fallback. Anti-enumerates: if user not found, still returns success.

POST `/accounts/api/update-profile/` and

POST `/accounts/api/change-password/` are straightforward auth-gated mutations. The change-password view ends with `update_session_auth_hash(request, user)`, which is the standard Django incantation to keep the existing session valid after changing the password (otherwise the user would be logged out instantly).

4.2.4 Admin

`UserAdmin(BaseUserAdmin)` (`accounts/admin.py:6–54`) exposes:

- `list_display`: email, first/last name, email_verified, token_balance, is_staff, date_joined.
- `list_filter`: is_staff, is_superuser, email_verified, date_joined.
- `search_fields`: email, first/last name, phone_number.
- Three custom actions: `add_100_tokens`, `add_1000_tokens`, `verify_users` (bulk update to `email_verified = True`).

These actions are how customer-support style operations happen: a staff user picks 200 accounts in the admin, hits *Add 100 tokens to selected users*, done.

4.3 App: companies

The company directory: who exists, what sector, what role, what products, what key contacts.

4.3.1 Models

`backend/companies/models.py` defines seven model classes plus an `auditlog` block:

- `Sector`, `CompanyRole`, `CompanyType` — three taxonomy tables. Each has `name + description`. Used by `Company` via `ForeignKey(SET_NULL)`.
- `Company` (lines 47–114) — the directory entity. 24 fields covering identity, location, contact info, business details, verification status, and bookkeeping. Two indexes: `(‘country’, ‘sector’)` and `(‘verification_status’,)`. Verification is a four-state choice: `unverified`, `pending`, `verified`, `pre`.
- `CompanyProduct` (lines 117–139) — products a company offers. `ForeignKey(Company, CASCADE)`, `name`, `description`, `variety` (e.g. “Basmati”, “Super Kernel”), `value_added` (e.g. “Organic certified”), `hsn_code`.
- `KeyContact` (lines 142–164) — a person at a company: `name`, `designation`, `phone`, `whatsapp`, `email`, `is_public=True` contacts are visible without spending a token.
- `KeyContactUnlock` (lines 167–191) — the “user X spent 1 token to see contact Y” ledger. `unique_together=[‘user’, ‘key_contact’]` prevents double-charging.

The auditlog block at the end registers `Company`, `CompanyProduct`, and `KeyContact`, so any change to those models lands in the third-party `auditlog_logentry` table with the user attribution that the middleware thread-local provides.

4.3.2 Serializers

`backend/companies/serializers.py` has eight serializers. The clever one is `KeyContactSerializer` (lines 39–72) which implements the paywall at the serialization layer:

```
60 def to_representation(self, instance):
61     rep = super().to_representation(instance)
62     if not (instance.is_public or rep['is_unlocked']):
63         rep['phone'] = rep['email'] = rep['whatsapp'] = "Locked"
64     return rep
```

If neither public nor unlocked-by-this-user, the contact fields are overwritten with the literal string `Locked` (with a lock emoji in the source). The frontend reads this string and renders the lock icon plus an “Unlock for 1 token” button.

4.3.3 Endpoints (`companies/views.py`)

Mounted at `/api/` (so `companies/` becomes `/api/companies/`, and so on).

M.	URL	What it does
GET	<code>/api/companies/</code>	List+filter directory; supports <code>?search=</code> , <code>?use_ai=</code> , <code>?region=</code> , <code>?sector=</code> , <code>?type=</code> , <code>?role=</code> .
GET	<code>/api/companies/<pk>/</code>	Detail, with nested products + key contacts.
GET	<code>/api/companies/regions/</code>	Distinct provinces.
GET	<code>/api/companies/countries/</code>	Distinct countries; supports <code>?role=</code> .
GET	<code>/api/companies/hsn_codes/</code>	Distinct HSN codes from <code>CompanyProduct</code> .
GET	<code>/api/key-contacts/</code>	List contacts; <code>?company=</code> .
GET	<code>/api/key-contacts/<pk>/</code>	Single contact (with paywall).
POST	<code>/api/key-contacts/<pk>/unlock/</code>	Spend 1 token; create <code>KeyContactUnlock</code> .
GET	<code>/api/sectors/</code>	Sectors lookup.
GET	<code>/api/company-types/</code>	Types lookup.
GET	<code>/api/company-roles/</code>	Roles lookup.

Table 4.3: `companies` endpoints.

`CompanyViewSet.get_queryset` — the smart-search hook (`companies/views.py:61-167`).

When you pass `?search=foo&use_ai=true`, the view:

1. Snapshots the first 100 candidates as a `values()` dict.
2. Calls `utils/ai_service.py:AIService.smart_search(search, candidates)` (a GPT-4o-mini wrapper).
3. Filters to whatever IDs GPT returned, preserving GPT's ordering with a `Case/When/IntegerField order_by`.
4. If GPT returns nothing or errors, sets `self._ai_fallback = True` and falls back to plain `Q(name__icontains=search) | Q(description__icontains=search)`. The list response then carries `ai_fallback: true` so the frontend can show “AI search unavailable, results from keyword fallback.”

`KeyContactViewSet.unlock` — the token transaction (`companies/views.py:212-270`). Atomic, CSRF-exempt, auth-required:

1. If the contact is already unlocked by this user → `\{"status": "already_unlocked"\}`.
2. If `contact.is_public` → create the unlock with `tokens_charged = 0`.
3. If user has no tokens → HTTP 402 with current balance.
4. Else `user.deduct_tokens(1)` and create the unlock; HTTP 200 with `tokens_charged = 1` and the now-unlocked contact serialized with the request context (so the frontend sees the real fields, not `Locked`).

4.3.4 Admin

Seven admin registrations. The most interesting:

- `CompanyAdmin` uses a custom `CompanyAdminForm` that swaps the description field to `CKEditor5Widget`. Six fieldsets: Basic Information, Description, Contact Information, Business Details, Status & Flags, Metadata.
- `KeyContactAdmin` adds a `KeyContactUnlockInline` table inline so you can see “who unlocked this contact” directly on the page. A custom `unlock_count` method counts the inline rows.

4.3.5 Management commands

Nine custom commands under `backend/companies/management/commands/`:

Command	What it does
<code>seed_roles</code>	Creates 5 baseline roles: Buyer, Supplier, Distributor, Manufacturer, Service Provider.
<code>setup_company_roles</code>	Creates 3 binary roles: Buyers, Suppliers, Both. Bulk-assigns every roleless company to “Both”.
<code>import_companies</code>	Reads <code>companies.xlsx</code> , walks rows, and <code>update_or_create</code> s a <code>Company</code> per row using a column-mapping dict. Wrapped in <code>transaction.atomic()</code> .
<code>debug_companies</code>	Diagnostic: prints counts, lists roles, simulates a viewset filter. No-op on data.
<code>merge_sectors</code>	One-shot: merges <code>Sector(id=3, 'Confectionary')</code> into <code>Sector(id=4, 'Confectionery')</code> and deletes the duplicate.
<code>fix_sectors_hsn</code>	Generalisation of the above plus auto-assigns a random plausible HSN code to any <code>CompanyProduct</code> that lacks one.
<code>fix_company_data</code>	Bulk-marks every company <code>verification_status='verified'</code> (a one-shot for the demo).
<code>analyze_companies</code>	For every company description, runs <code>utils/ai_service.py:analyze_sentiment</code> and stores “Positive”/“Negative”/“Neutral” in <code>market_sentiment</code> .
<code>generate_embeddings</code>	For each verified company, calls <code>utils/ai_service.py:get_embedding</code> (OpenAI <code>text-embedding-3-small</code> , 1536-d) and stores in <code>trade_data.CompanyEmbedding</code> .
<code>index_companies</code>	Builds an HNSW vector index in Redis (<code>idx:companies</code>) for Smart Search.

Table 4.4: `companies` management commands.

4.4 App: subscriptions

The token-balance economy: plans, redeem codes, purchases, and the unified payroll service.

4.4.1 Models

Five model classes in `backend/subscriptions/models.py`:

- `SubscriptionPlan` — `plan_name`, `price`, `currency`, `tokens_included`, `description`, `features` (a `JSONField` for feature flags).
- `UserSubscription` — the active subscription record. `ForeignKey(plan, PROTECT)` ensures you cannot delete a plan that still has subscribers.
- `TokenPurchase` — one-off token top-up record with payment provider/reference fields (PayPal, Stripe, JazzCash, etc.).
- `RedeemCode` — a 12-character alphanumeric code (with confusable characters `0/O/I/1` stripped) that can be redeemed by exactly one user. Methods: `generate_code(length=12)` and `redeem(user)`.
- `UserAccess` — the entitlement ledger. Two tiers: `HS_CODE` (full category access, costs 5,000 tokens) and `PRODUCT` (single subcategory, 500 tokens). `UniqueConstraint(['user', 'hscode', 'subcat_id'])` prevents double-grants.

4.4.2 The payroll service: `subscriptions/services.py`

This module is the *single backend authority* for “does this user have access?” Constants (lines 25–32):

```

25 HS_CODE_PRICE = 5000
26 PRODUCT_PRICE = 500
27 FULL_ACCESS = 'FULL_ACCESS'
28 PRODUCT_ACCESS = 'PRODUCT_ACCESS'
29 NO_ACCESS = 'NO_ACCESS'

```

`get_access_state(user, hscode, subcat_id=None)` (lines 35–60). Returns one of the three states. Logic:

1. Anonymous → `NO_ACCESS`.
2. Staff/superuser → `FULL_ACCESS` (admin bypass).
3. Look up `UserAccess` rows for this user + this hscode.
4. Any `HS_CODE` row → `FULL_ACCESS`.
5. Else if `subcat_id` matches a `PRODUCT` row → `PRODUCT_ACCESS`.
6. Else → `NO_ACCESS`.

`purchase_access(user, access_type, hscode, subcat_id=None)` (lines 63–108). Returns (success: bool, message: str, new_balance: Optional[int]). Inside a `transaction.atomic()` it:

1. Validates inputs and computes the price.
2. *Idempotency*: if the grant exists, returns `(True, "Access already granted.", balance)` without charging.
3. Checks `token_balance >= price`; otherwise fails.
4. Deducts and creates the `UserAccess`.

4.4.3 Endpoints

M.	URL	Effect
GET	<code>/api/subscriptions/plans/</code>	Lists all plans (no auth).
POST	<code>/api/subscriptions/redeem/</code>	“Demo bypass” — credits the plan’s tokens directly to <code>request.user.token_balance</code> (does not validate a real code). Auth required.
POST	<code>/api/subscriptions/purchase-access/</code>	Calls <code>services.purchase_access</code> . Auth required.
GET	<code>/api/subscriptions/profile-data/</code>	Returns <code>\{entitlements, tokenHistory, telemetry\}</code> for the profile page. Auth required.
POST	<code>/api/subscriptions/generate-codes/</code>	Staff-only: bulk-creates 10 redeem codes for a given plan. Form-encoded.

Table 4.5: subscriptions endpoints.

`profile_data_view` is rich enough to deserve a walkthrough (`subscriptions/views.py:145-255`). It returns a single payload that the React profile page renders into three tabs:

1. **Entitlements.** For each `UserAccess` row, derives a name (`ProductCategory.hs_code` for `HS_CODE`, `ProductSubCategory.id` for `PRODUCT`) and an `accessLevel` string.
2. **Token history.** A union of three event streams: `UserAccess` (negative amount = spend), `TokenPurchase` (positive = top-up), `RedeemCode` (positive = redeem). Sorted descending by date.
3. **Telemetry.** Two derived fields: `queries = total_tokens_spent // 10 + 12` and `productsViewed = total_tokens_spent // 10 + 12`. These are loose proxies (the real per-user query counter is not stored).

4.4.4 Admin

Four admin classes; the highlight is `RedeemCodeAdmin` which adds six bulk-generate actions, one per plan, each emoji-labelled. Selecting the action seeds 15 fresh codes in one click. The status badge column uses `format_html` to render coloured pills (green/gray/red).

4.4.5 Management commands

- `create_plans` — seeds the six canonical plans (see Table 4.6).
- `generate_codes` — CLI alternative to the admin bulk action; resolves a plan by id or fuzzy name match, then generates a configurable number of codes.

Plan name	Price	Currency	Tokens
500 Credits	9.00	PKR	500
5K Credits	32.00	PKR	5,000
15K Credits	99.00	PKR	15,000
6000 Credits Annually	99.00	PKR	6,000
60K Credits Annually	350.00	PKR	60,000
900K Credits Annually	1090.00	PKR	900,000

Table 4.6: Subscription plans seeded by `create_plans`.

4.5 App: `trade_data`

The most data-heavy app. Owns the trade product hierarchy and the `Transaction` ledger, plus two embedding tables. Its `views.py` and `admin.py` are intentionally empty — this is a model + management-command layer that other apps query directly.

4.5.1 The product hierarchy

A four-level tree on top of HS codes:

```
Product (HS chapter, e.g., 1701)
  -> ProductCategory (HS heading, e.g., 1701.99)
    -> ProductSubCategory (e.g., 1701.9990 "Refined Sugar")
      -> ProductItem (e.g., "White Refined Sugar 50KG Bag")
```

Listing 4.1: Product hierarchy

Each level is a Django model with a `ForeignKey(parent, on_delete=CASCADE)`. The leaf, `ProductItem`, is what every `Transaction` points at via a `ForeignKey(SET_NULL)`.

4.5.2 The `Transaction` model

The table that everything else is built on. 21 fields covering source, direction, parties, geography, quantity, price, and bookkeeping. Key fields (`trade_data/models.py:65-142`):

Seven indexes on `Meta.indexes` (lines 121–133) cover every hot filter the search engine uses: `reporting_date`, `buyer`, `seller`, `hs_code`, `trade_type`, `origin_country`, `destination_country`.

Why This Matters

The model keeps both *kg* and *MT* (metric tonne) columns even though one is just 1,000× the other. Two reasons: (a) the source spreadsheets sometimes give one but not the other, and (b) aggregation queries like `SUM(qty_mt)` are cheaper to read than to compute on the fly.

4.5.3 Embedding tables

Two thin wrappers around vectors stored as JSON:

- `CompanyEmbedding` — `company_name` (unique), `embedding` (`JSONField`), `cluster_tag`, `pagerank`, `degree`.

Field	Type	Notes
trade_type	CharField, choices	IMPORT / EXPORT , db-indexed.
hs_code	CharField(50)	db-indexed.
product_item	ForeignKey(SET_NULL) to ProductItem .	
buyer/seller/shipp	CharField(500)	free-text.
origin_country	CharField(100)	db-indexed.
destination_countr	CharField(100)	db-indexed.
qty_kg/qty_mt	DecimalField(20,6)	both kept; the ingest derives one from the other when one is missing.
usd_per_kg/usd_per	DecimalField(20,6)	nullable.
pkr/usd	DecimalField(30,2)	total invoice value.
reporting_date	DateField	db-indexed for time queries.

Table 4.7: Highlights of the Transaction model.

- ProductEmbedding — ForeignKey(ProductItem) (unique), embedding , cluster_tag .

These are populated by generate_embeddings (companies) and generate_gnn_embeddings (referenced from the setup script).

4.5.4 Ingestion commands

- ingest_trade -file <xlsx> — IMPORT direction. Reads the Excel header at row 6 (so spreadsheet skipping happens automatically), normalises column names, formats HS codes to 4-decimal strings, walks the product hierarchy with get_or_create , and bulk-inserts Transaction rows with ignore_conflicts=True .
- ingest_trade_export -file <xlsx> — EXPORT direction; identical except origin_country = "Pakistan"
- repair_trade_companies — backfills the Company table from existing Transaction rows. Iterates with .iterator() to avoid loading all into memory; get_or_create s a directory entry per unique buyer/seller name.

4.6 App: search (API surface)

We cover the heavy ML inside this app in Chapter 5. Here we focus on the API surface (apps.py , models.py , urls.py , views.py , the management commands).

4.6.1 Boot warmup (search/apps.py)

search/apps.py contains a small but important ready() hook:

```

1 class SearchConfig(AppConfig):
2     default_auto_field = 'django.db.models.BigAutoField'
3     name = 'search'
```

```

4     def ready(self):
5         def _warmup():
6             try:
7                 from .services.nlu_engine import ModernNLUEngine
8                 engine = ModernNLUEngine()
9                 engine.parse("buy sugar")
10                engine.parse("sell wheat to UAE")
11                print("[ZaraiLink] NLU warmup complete - system ready.")
12            except Exception as e:
13                print(f"[ZaraiLink] Warmup skipped: {e}")
14            threading.Thread(target=_warmup, daemon=True).start()

```

Listing 4.2: NLU warmup

When Django boots, a daemon thread runs two throwaway NLU parses. That loads SetFit, GLiNER, and the SentenceTransformer once, so the first user query does not eat a 5-second cold-start.

4.6.2 The viewset and its endpoints

`backend/search/views.py` (892 lines) is built around a single `SearchViewSet(viewsets.ViewSet)` with `permission_classes = [AllowAny]`. A module-level singleton-pattern initialises the heavy `SearchService` once at import time:

```

1 logger.info("Initializing Global SearchService on Startup...")
2 global_search_service = SearchService()

```

Listing 4.3: Singleton SearchService

Every viewset instance reuses the same `SearchService`.

M.	URL	Action
GET	<code>/api/search/</code>	main NL search; supports paywall preview
GET	<code>/api/search/autocomplete/</code>	8-row product suggestions
GET	<code>/api/search/hs-code-tree/</code>	HS-code tree drill-down
GET	<code>/api/search/hs-summary/</code>	summary by HS or by word
GET	<code>/api/search/hs-dashboard/</code>	analytics dashboard for an HS prefix
GET	<code>/api/search/supplier-detail/</code>	full supplier profile (auth)
GET	<code>/api/search/compare/</code>	multi-supplier comparison (auth)
GET	<code>/api/search/supplier-transactions/</code>	paginated transaction list (auth)

Table 4.8: search endpoints.

GET `/api/search/` — main NL search

Query params: `q` (required), `scope` (`import` / `export`, default `import`), optional `hs_code`, `subcat_id`, `varian`

The view's first move is to extract a top-N hint ("top 5 suppliers") via the regex `r'\textbackslash b(?:top|best|1`

Then it delegates to `search_service.execute_search(...)`. The response can be one of five shapes:

1. **Category bridge** — “the query maps to a single category, do you want to see it?”
2. **No data** — “no transactions match.”
3. **Scope mismatch** — “you searched in IMPORT scope but the data is in EXPORT.”
4. **Needs disambiguation** — “which variant did you mean: Refined / Raw / Brown sugar?”
5. **Normal results** — the supplier list.

For normal results, the view computes a volume-weighted average price and a market snapshot, then enforces the token payroll:

```
access = get_access_state(request.user, active_hs_code, active_subcat_id)
if access not in (FULL_ACCESS, PRODUCT_ACCESS):
    visible = visible[:2] # only show 2 of N suppliers
paywall_price = PRODUCT_PRICE if active_subcat_id else HS_CODE_PRICE
```

Listing 4.4: Paywall preview slicing

The 2-supplier preview gives the user a taste; unlocking takes either 500 (single product) or 5,000 (full HS category) tokens.

GET /api/search/autocomplete/

Returns at most 8 suggestions of the form `\{label, name, hs_code, category, total_volume, tx_count, is_final\}`.

The `label` is human-formatted: e.g. “Dextrose Anhydrous (12,345 MT)”.

GET /api/search/hs-code-tree/

A tree-explorer for HS codes. The behaviour depends on the length of the sanitised query:

- ≤ 2 chars: chapter level. Groups `Transaction` by the first 4 digits of `hs_code`.
- 3 chars: still 4-digit headings.
- 4–7 chars: queries `ProductCategory.filter(hs_code__startswith=q)` for distinct categories. `is_final: True`.
- 8+ chars: exact tariff match, max 5 rows.

GET /api/search/hs-summary/

A combined summary view that branches on whether `q` is alpha or numeric. Alpha: word-search across product subcategory *names*. Numeric: same drill-down logic as `hs-code-tree` but returning counts. A nested helper, `get_contextual_name`, prepends the parent category name when the leaf name is vague (“other”, “other items”, “not elsewhere specified”).

GET /api/search/hs-dashboard/

The analytics endpoint behind the `DataDashboard.js` page. Query params: `q` (HS-code prefix), `intent` (one of `FOREIGN_SUPPLIERS`, `PAKISTANI_BUYERS`, `FOREIGN_BUYERS`, `PAKISTANI_SUPPLIERS`, default `FOREIGN_SUPPLIERS`), optional `subcat` comma-list.

The view:

1. Builds a `Transaction.filter(hs_code__startswith=q)` base queryset.

2. Builds a sidebar by grouping on `product_item__sub_category__name`, deduping by name.
3. Optionally applies the `subcat` filter; if it would yield zero rows, silently reverts and sets `filter_was_ignored = True`.
4. Filters by trade type per the intent.
5. Aggregates per buyer/seller + per origin/destination country: `Sum(qty_mt), Avg(usd_per_mt), Count(id)`.
6. Returns sidebar counts *and* 50 rows of profiles, with the same payroll-preview slicing as the main search endpoint.

GET /api/search/supplier-detail/

Auth required. Resolves intent (URL > NLU cache > parse), resolves subcategories, calls `get_access_state`; if access denied returns HTTP 403. Otherwise routes to `aggregator.get_buyer_details(...)` (intent=SELL) or `aggregator.get_supplier_details(...)` (intent=BUY).

Response:

```
{
  "supplier": { ...rich detail... },
  "comparables": [],
  "trade_lens_product_id": null,
  "type": "SUPPLIER" | "BUYER"
}
```

GET /api/search/compare/

Auth required. Comma-separated `?suppliers=A,B,C`. Same auth & access checks, then delegates to `aggregator.get_supplier_comparison(...)`.

GET /api/search/supplier-transactions/

Auth required. The transaction list behind the `DealDetail.js` page's Transactions tab. Supports paging (`page`, `page_size=15`) and filters (`start_date`, `end_date`, `buyer`, `seller`, `country`, `min_qty`, `max_qty`).

4.6.3 Search management commands

- `build_search_index` — builds `backend/search_index.pkl` from `ProductSubCategory` rows. Loads `SentenceTransformer('all-MiniLM-L6-v2')`, encodes every name, and pickles a dict `\{ids, names, hs_codes, embeddings\}`.
- `sync_search` — bulk-syncs every `Transaction` into an OpenSearch index named `trade_index` with a KNN-enabled mapping. Configurable via the constants at the top of `sync_search.py`: `OPENSEARCH_HOST`, `INDEX_NAME`, `VECTOR_DIM=384`, `EMBEDDING_MODEL='all-MiniLM-L6-v2'`, `BATCH_SIZE=500`. The OpenSearch path is currently *disabled* in the live runtime via an `if False` and ... guard at `search_service.py:343`; the team kept the indexer in tree as the migration target.

4.7 Empty placeholder apps

- `backend/home/` — `models.py`, `views.py`, `admin.py`, `tests.py` are all skeleton (one Django default import each). Not in `INSTALLED_APPS`.
- `backend/trade_directory/` — same shape.

These exist as reservations for future expansion; the work moved into existing apps. Do not be surprised when `grep` finds nothing in them.

4.8 The utils package

Two helper modules, both class-based facades.

4.8.1 `utils/redis_client.py`

A static-method facade around `redis-py` for the Redis vector index used by Smart Search:

- `create_index(index_name='idx:companies', dim=1536)` — creates an HNSW vector index in Redis with cosine distance.
- `index_data(prefix, doc_id, embedding, metadata)` — `HSET` s a hash with the embedding bytes plus metadata.
- `vector_search(query_embedding, k=10)` — KNN search.
- `cache_set(key, value, ttl=86400)` and `cache_get(key)` — thin wrappers used by the NLU cache.

4.8.2 `utils/ai_service.py`

A facade around the OpenAI client. Three classmethods:

- `get_embedding(text)` — calls `client.embeddings.create(model='text-embedding-3-small', input=text)` returns the 1536-d float vector or `None` on failure.
- `smart_search(query, candidates)` — prompts `gpt-4o-mini` with the candidate company list and asks it to rank them by relevance. Returns the IDs in GPT's order, or `None` on parse failure.
- `analyze_sentiment(text)` — a one-shot prompt to classify a description as “Positive”/“Neutral”/“Negative”.

If `OPENAI_API_KEY` is empty, every call returns `None` and the calling code falls back to keyword-only behaviour.

4.9 Test configuration

`backend/conftest.py` (392 lines) is the `pytest` fixture factory — 28 fixtures plus 2 helpers. Highlights:

- User fixtures: `user_factory`, `user`, `authenticated_user`, `staff_user`, `superuser`.

- Company fixtures: `sector`, `company`, `verified_company`, `key_contact`, `public_key_contact`.
- Subscription fixtures: `subscription_plan`, `redeem_code`, `token_purchase`.
- Trade fixtures: `transaction_factory`, `import_transaction`, `export_transaction`.
- API client fixtures: `api_client`, `authenticated_client`.

```
pytest.ini sets DJANGO_SETTINGS_MODULE = zarailink.settings, python_files = test_*.py *_test.py,
python_classes = Test*, python_functions = test_*, and addopts = -reuse-db -p no:cacheprovider.
```

We dedicate Chapter 9 to actually *running* the suite and writing new tests.

4.10 Endpoint reference summary

The complete enumeration of HTTP endpoints is in Appendix B. As a quick mental model, you can think of the URL space as five regions:

- `/admin/...` — Django admin.
- `/accounts/...` — auth.
- `/api/companies/...`, `/api/key-contacts/...`, `/api/sectors/...` etc. — directory.
- `/api/subscriptions/...` — token economy.
- `/api/search/...` — the search engine surface.

Anything else — catch-all `re_path` — is the React SPA.

Chapter 5

The Search Engine and the ML Pipeline

This is the longest and most novel chapter of the guide because the search engine is what makes Zarailink *Zarailink*. We start with the high-level shape and then drill into each stage with code. By the end you should be able to debug a bad search result by tracing it through every class.

5.1 What the search engine has to do

The job sounds simple: “user types a query, system returns ranked suppliers / buyers.” The reality is harder because the input space is unbounded:

- Queries vary from “1701” (an HS code) to “*cheap basmati rice from Punjab to Dubai with at least 50 MT and verified suppliers only*”.
- Vocabulary is messy: “Pak”, “UAE”, “KSA”, “Brasil”, misspellings (“brzail”), abbreviations.
- Intent is implicit: “*best importers of edible salt*” is a BUY query about *exporters* (importer = the user, exporter = the entity being looked for); “*cheapest sugar*” is the same BUY but with a price-ascending ranking hint.
- Aggregation is heavy: each query rolls up potentially millions of `Transaction` rows by supplier and computes volume-weighted prices, frequencies, recency.
- Ranking must use both content and behaviour signals: a 0.7 heuristic + 0.3 LightGBM model.
- And it must all happen in well under two seconds.

5.2 The four-stage pipeline

Every search query traverses four logical stages. Figure 5.1 shows the full flow.

We will visit each stage in turn.

5.3 Stage 1 — The NLU engine

5.3.1 The two parsers, side by side

There are *two* natural-language parsers in the codebase. Knowing the distinction is essential to avoid debugging the wrong file.

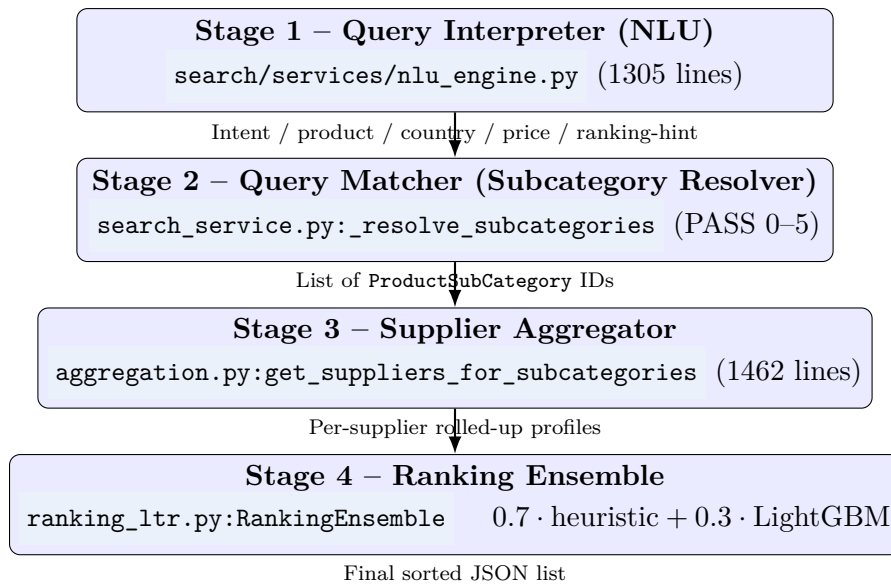


Figure 5.1: The four-stage search pipeline.

- `backend/search/services/nlu_engine.py` — `ModernNLUEngine`, the production parser. ML-driven (SetFit + GLiNER + RapidFuzz). 1,305 lines.
- `backend/search/services/query_parser.py` — `QueryInterpreter`, the legacy v2 parser. Pure rules (regex + dictionaries). 378 lines. Still used by `candidate_retrieval.py` and the LTR dataset builder (`ltr_dataset_builder.py`). It also defines the *9-family taxonomy* that feeds the ranker’s per-family weights.

The runtime path uses `ModernNLUEngine`; the legacy parser is quoted only when its 9 families are referenced.

5.3.2 What `ModernNLUEngine.parse` returns

A nested dict whose keys the rest of the pipeline consumes. The shape:

```

1 {
2   "intent":          "BUY",    # or "SELL" or "UNKNOWN"
3   "product_keyword": "basmati rice",
4   "country":         "United Arab Emirates",
5   "price_filter":    {"range": {"usd_per_mt": {"lte": 400.0}}},
6   "volume_filter":   {"qty_mt": {"gte": 50.0}},
7   "ranking_hint":    "price_asc",
8   "scope":           "WORLDWIDE",
9   "raw_query":       "cheap basmati rice from Punjab to UAE under $0.40/kg ↵
10   with at least 50MT",
11 }

```

Listing 5.1: Sample NLU output

5.3.3 Inside the engine, step by step

The class lives at `nlu_engine.py:611+`. `parse(query, ui_context='worldwide')` runs five logical sub-stages:

Stage 1.1: country catalog and aliases. `nlu_engine.py:42-72` builds `_COUNTRY_CATALOG` from `pycountry` (every country indexed by name, alpha-2, alpha-3, and common name) plus a hand-maintained `_MANUAL` alias dict (`pak` → Pakistan, `uae` → UAE, `ksa` → Saudi Arabia, `korea` → South Korea, etc.). A second variable, `_COUNTRY_NAMES_ONLY`, is the fuzzy-matching corpus — only full names go in, so a typo like `chna` can never collide with `che` (Switzerland’s alpha-3 code).

Stage 1.2: ranking-hint detection. `nlu_engine.py:122-143` walks an ordered table of phrases (long phrases first to avoid “*top quality sugar*” resolving to `volume_desc` from the bare “top”). Yields one of `price_asc`, `price_desc`, `volume_desc`, `reliability`, or `None`.

Worked Example

```
"need cheap sugar" → price_asc . "premium dextrose from germany" → price_desc .
"biggest buyers of sugar" → volume_desc . "serious buyers only" → reliability .
"buy sugar from brazil" → None .
```

Stage 1.3: noise stripping. `strip_noise_tokens(keyword)` (lines 160–187) defines a 21-element noise list (`export`, `import`, `buy`, `sell`, `cheap`, `bulk`, `urgent`, `fast`, `suppliers`, `buyers`, ...) and drops any token whose RapidFuzz `WRatio` score against the list is ≥ 80 . Single-word keywords are sacrosanct (`"dextrose"` stays as `"dextrose"`). Lazy import of RapidFuzz so this stays cheap.

Stage 1.4: intent classification (SetFit). The trained SetFit model (a sentence-transformer body `paraphrase-MiniLM-L6-v2` plus a sklearn `LogisticRegression` head) lives at `backend/models/zarai_inten`. Loaded once by the engine, it answers the question: *is this a BUY query or a SELL query?*

Why This Matters

Why SetFit and not a fine-tuned BERT? SetFit is a contrastive fine-tuning approach designed for tiny labelled sets (here, 30 hand-written seeds at `train_intent.py:43-82`). It avoids the full classifier fine-tuning loop while still beating zero-shot baselines. For BUY/SELL, that is more than enough.

Stage 1.5: entity extraction (GLiNER). GLiNER is a zero-shot NER model. The engine asks it to find spans typed as `country`, `product`, `quantity`, `price`. Extraction failure is treated as “no signal,” not as an error.

Stage 1.6: country resolution (RapidFuzz). Three layers (`nlu_engine.py:1199-1260`):

1. *Preposition patterns*: regex like `from <X>`, `based in <X>`, `within <X>`.

2. *Has-preposition guard*: if the query contains a preposition, skip the free-token scan to avoid spurious short-code hits.
3. *Free-token scan*: each word ≥ 4 chars gets an exact catalog lookup. Forbidden prefixes (`where`, `anywhere`, `maybe`, `idk`, `not sure`) are rejected.

Stage 1.7: price logic. `nlu_engine.py:317` (`_detect_price_operator`) scans for the LTE/GTE phrase sets:

- LTE: `under`, `below`, `max`, `less than`, `cheaper than`, `at most`.
- GTE: `above`, `over`, `min`, `more than`, `at least`, `minimum`.

Combined with regex captures of the numeric value and currency. If the regex path fails on a complex sentence, the engine falls back to an OpenRouter LLM call (`deepseek/deepseek-chat` , with `mistralai/mistral-7b-instruct:free` as a safety net) for structured extraction. The fallback is gated by `OPENROUTER_API_KEY` ; if absent, the engine simply returns no price filter.

5.3.4 The 9-family taxonomy

The legacy parser `QueryInterpreter` bins each query into one of nine families based on what filters were extracted (`query_parser.py:364-376`):

F.	Name	Trigger
1	Discovery / Generic	no filters; “find sugar suppliers”
2	Country-Filtered	at least one country in the query
3	Volume-Aware	explicit quantity (e.g., “50 MT”)
4	Price-Constrained	explicit price ceiling/floor
5	Time-Constrained	explicit time range (“last 6 months”)
6	Recommendation	“top”, “best”, “rank” phrasing
7	Market Intelligence	“compare”, “vs”, “cheapest”
8	Evidence	“shipments”, “transactions”, “record”
9	Hybrid	multi-intent queries (“and”, “;”)

Table 5.1: 9-family taxonomy and triggers.

The family number is later used by the heuristic ranker (`ranking_ltr.py:26-46`) to select a per-family weight vector.

5.3.5 NLU caching

Every parsed result is cached in Django’s cache backend (Redis or LocMem) with key `nlu_<md5(query)>` and 24-hour TTL (`search_service.py:219-228`). On a Redis-backed deploy, the cache is shared across worker processes, so the same query parsed by worker A benefits worker B for free.

5.4 Stage 2 — Subcategory resolver

The interpreter gave us `product_keyword="basmati rice"` . We now need to translate that string into a list of `ProductSubCategory` IDs that the SQL aggregation can filter on. This happens in

`search_service.py:_resolve_subcategories` (lines 409–790).

5.4.1 The five passes

The resolver runs up to five passes *in order* and stops at the first that returns rows:

PASS 0 — Category match. `ProductCategory.objects.filter(name__icontains=keyword)` .

Pulls every subcategory under matching categories. Catches “rice” → all rice subcategories.

PASS 1 — Subcategory prefix match. `ProductSubCategory.objects.filter(name__startswith=keyword)`

PASS 2 — Item prefix match. `ProductItem.objects.filter(name__startswith=keyword)` .

PASS 3 — Trigram fuzzy. `TrigramSimilarity` from `django.contrib.postgres.search` , threshold > 0.45 , top 5 each on subcategories and items.

PASS 4 — Word-by-word. Splits the keyword into words longer than 2, sorts longest-first, then runs PASS 1 + PASS 3 per word.

PASS 5 — RapidFuzz fallback. Loads the in-process catalog (one DB read per worker) and runs `process.extractOne(keyword, catalog, scorer=fuzz.WRatio, score_cutoff=75)` . If it finds a corrected name, re-runs PASS 1 with the corrected form.

Why This Matters

The cascade is intentionally cheap-first. PASS 0 is a single ORM query; PASS 5 loads the entire product catalog into memory. Most queries resolve at PASS 0 or PASS 1. Misspellings fall through to PASS 3–5.

5.4.2 HS-code drill-down

If the user supplied an explicit HS code (numeric query, autocomplete selection, etc.), the resolver

short-circuits all five passes and walks the HS hierarchy via `ProductSubCategory.filter(hs_code__startswith=...)` (`search_service.py:459-576`). The behaviour depends on the prefix length:

- ≤ 3 chars → chapter level, 4-char prefix buckets.
- $= 4$ chars → heading → 7-char prefix buckets.
- ≥ 5 chars and many results → group by unique product names.
- Few results → list each subcategory directly.

5.4.3 Scope-aware availability filter

After candidate IDs are found, the resolver verifies that the active scope (`IMPORT` / `EXPORT`) actually has data for those IDs (`search_service.py:690-789`). If the active scope is empty but the opposite scope has data, the resolver *flags a scope mismatch*: the API responds with a banner like “*We have export data for sugar from India. Did you mean to switch to export?*” and waits for the user to confirm. This is what stops the system from returning “no results” for a query that has plenty of data on the other side.

5.4.4 Disambiguation

If the same name maps to multiple distinct subcategories (e.g. “Sugar” matches Refined, Raw, and Brown), the resolver returns `needs_disambiguation: True` with a list of variants. The

view relays that to the frontend, which renders a variant picker.

5.5 Stage 3 — Supplier aggregator

The resolver gave us a list of subcategory IDs. Now we ask the database “which companies traded those products and how much.” This is `aggregation.py:SupplierAggregator`, the largest single class in the project (1,462 lines).

5.5.1 The base aggregation

`get_suppliers_for_subcategories` is the entry point. The heart of it is a single Django ORM aggregation:

```
1 qs = Transaction.objects.filter(...)
2 qs = qs.values(target_field, country_field).annotate(
3     total_volume = Sum('qty_mt'),
4     weighted_price_sum = Sum(F('qty_mt') * F('usd_per_mt'),
5                             output_field=FloatField()),
6     shipment_count = Count('tx_reference', distinct=True),
7     last_shipment_date = Max('reporting_date'),
8     max_shipment_vol = Max('qty_mt'),
9     avg_shipment_vol = Avg('qty_mt'),
10 ).order_by('-total_volume')
```

Listing 5.2: The supplier aggregation kernel

The volume-weighted average is computed in Python afterwards as `avg_price = weighted_price_sum / total_volume`.

This is critical: a plain `Avg('usd_per_mt')` would be biased toward small shipments.

5.5.2 The intent-and-scope matrix

Which fields you group by depends on *both* the user’s intent (`BUY` or `SELL`) and the scope (`IMPORT` or `EXPORT`):

Scope	Intent	Group by entity	Group by country
IMPORT	BUY	seller	origin_country
IMPORT	SELL	buyer	destination_country
EXPORT	SELL	buyer	destination_country
EXPORT	BUY	seller	origin_country

Table 5.2: Intent-and-scope routing inside the aggregator.

5.5.3 Filters

- **Country:** `f"\{country_field\}__in"`.
- **Time:** `reporting_date__gte/lte`.
- **Price:** applied *post*-aggregation on `avg_price`, not on individual transactions. The comment at `aggregation.py:114-119` explains: “dropping a supplier just because one of

their shipments was above the ceiling” would be brittle; instead the aggregator drops suppliers whose *weighted average* fails the price predicate. This is equivalent to a SQL `HAVING` clause.

- **Volume:** similarly post-applied. If the user wants ≥ 100 MT, the aggregator computes a `volume_score`:

$$\text{score} = 0.5 \cdot \min\left(\frac{\text{single}}{V}, 1\right) + 0.3 \cdot \min\left(\frac{\text{capacity}}{V}, 1\right) + 0.2 \cdot \min\left(\frac{\text{avg}}{V}, 1\right)$$

and a `volume_fit` bucket: Strong (≥ 0.8), Good (≥ 0.5), Partial (≥ 0.3), Low.

5.5.4 The detail and intelligence helpers

`get_supplier_details(seller_name, ...)` (lines 209–347) and `get_buyer_details(buyer_name, ...)` (lines 349–485) compose the `DealDetail.js` page. They return:

- **Headline stats:** total volume, average price, shipment count, last shipment date.
- **Sparkline** — monthly buckets via `TruncMonth` with `Sum(qty_mt)` and `Avg(usd_per_mt)`; only months with positive price survive.
- **History** — last 50 transactions.
- **Filters** — distinct destination/origin countries.
- **Shipment-size buckets** — `Case/When` into 0-25, 25-50, 50-100, 100+ MT buckets, with count and average price per bucket.
- **Buyer/seller insights** — unique counterparty count and a 30-day-active count.
- **Intelligence** — repeat ratio, concentration ratio, pricing label, momentum label (see below).
- **Overview / market pricing / company profile** — detailed sub-objects rendered into the four `DealDetail` tabs.

Intelligence labels

`_calculate_intelligence` (lines 736–832) emits four labels:

- **Repeat ratio.** % of transactions accounted for by counterparties with > 1 deal. Strong (> 70), Moderate (> 30), Low (≤ 30).
- **Concentration ratio.** Top-3 counterparty volume / total. High (> 60), Moderate (> 30), Low.
- **Pricing label.** Combination of price volatility ($\sigma/\bar{p}$) and trend (latest vs earliest):
 - Buyer perspective: Opportunistic / High Sensitivity / Stable Procurement.
 - Supplier perspective: Premium / Competitive / Stable.
- **Momentum.** Volume(last 90d) vs Volume(prev 90d). Growing ($> 10\%$), Stable, Declining.

These labels are what give the `DealDetail` page its narrative quality (“Premium pricing with Growing momentum”).

Overview, market-pricing, company-profile

`_compute_overview_metrics` (lines 487–734), `_compute_market_pricing` (lines 937–1111), and `_compute_company_profile` (lines 1113–1347) each return rich nested dicts. Highlights:

- **Frequency pattern** — ratio `active_months / span` bucketed into Monthly Consistent (≥ 0.8), Regular (≥ 0.5), Sporadic.
- **Volume trend** — compares first vs second half of months, emits text like “+14% over period”.
- **Geography summary** — the top-5 origin/destination countries, paraphrased into sentences like “*Primarily Brazil-sourced with secondary suppliers in India and Thailand*”.
- **Top counterparties** — top 5 + “+N other” overflow row; Diversified / Moderate / Concentrated label.
- **Year-over-year growth** — $> 5\%$ Growing, $< -5\%$ Declining, else Stable.
- **Concentration buckets**: top-5-share > 80 Highly Concentrated, > 50 Moderately Diversified, else Highly Diversified.

5.6 Stage 4 — Ranking ensemble

The aggregator gave us a list of supplier dicts. Now we score and sort.

5.6.1 The 70/30 split

The ensemble is in `ranking_ltr.py:RankingEnsemble` (lines 206–264). The formula:

$$\text{final_score}(i) = 0.7 \cdot \text{heuristic}(i) + 0.3 \cdot \text{LightGBM}(i)$$

Both terms operate on the same eight-feature vector; the heuristic side uses a fixed, family-aware weight vector while the LightGBM side learns its weights via LambdaRank.

5.6.2 The eight features

`FeatureExtractor.FEATURE_NAMES` (`ranking_ltr.py:54-57`):

Feature	What it captures	How it is computed
<code>log_volume</code>	total tonnage on log scale	$\ln(1 + \text{total_volume_mt})$
<code>log_price</code>	average USD/MT on log scale	$\ln(1 + \text{avg_price})$, used with <i>negative</i> weight in family 4
<code>shipment_freq</code>	raw shipment count	<code>float(num_shipments)</code>
<code>inv_recency</code>	higher = more recent	$1/(\text{days_since_last_trade} + 1)$
<code>volume_fit_score</code>	how well capacity matches the requested volume	Strong=3, Good=2, Partial=1, Low=0
<code>scope_match</code>	always 1.0 today	retrieval already enforces scope; reserved
<code>country_match</code>	user’s country filter present?	1.0 if in filter, 0.5 if no filter, 0.0 otherwise
<code>price_fit</code>	is the avg price under the user’s ceiling?	1.0 yes, 0.5 no ceiling, 0.0 over

Table 5.3: The eight LTR features.

5.6.3 The family-weight table (heuristic side)

`ranking_ltr.py:26-46` defines per-family weight vectors. The intuition: each family of query cares about different things.

F. Active feature weights	
1	volume_fit 1.5, log_volume 1.0, shipment_freq 1.0, inv_recency 1.0, country_match 1.0
2	country_match 3.0, volume_fit 1.0, log_volume 1.0, inv_recency 0.5
3	volume_fit 3.0, log_volume 1.0, inv_recency 0.5
4	price_fit 3.0, log_price -1.0, volume_fit 1.0
5	inv_recency 3.0, shipment_freq 1.5, volume_fit 1.0
9	default hybrid: volume_fit 1.0, log_volume 1.0, inv_recency 1.0, shipment_freq 1.0, country_match 1.0

Table 5.4: FAMILY_WEIGHTS (heuristic ranker).

Note the *negative* weight on `log_price` for family 4 (price-constrained queries): everything else equal, cheaper is better.

5.6.4 The LightGBM LambdaRank model

Training (`train_ltr.py`).

```

47 params = {
48     'objective':      'lambdarank',
49     'metric':         'ndcg',
50     'ndcg_eval_at':   [5],
51     'learning_rate':  0.05,
52     'num_leaves':     31,
53     'verbose':        -1,
54 }
```

Listing 5.3: LightGBM hyperparameters

The trainer:

1. Calls `LTRDatasetBuilder.build_dataset(limit_per_product=50)` to get `(X, y, groups)`.
2. Splits 80/20 *group-aware* (so a query never crosses train/val).
3. Builds an `lgb.Dataset(X_train, label=y_train, group=train_groups)`.
4. Trains for up to 100 rounds with `early_stopping(10)` on validation NDCG@5.
5. Saves to `backend/search/models/lgbm_ltr.txt`.

Pseudo-labels via quantile binning. `ltr_dataset_builder.py:137-157` bins the heuristic teacher score by quintile to produce 5-grade labels:

```

df['pct'] = df['score'].rank(pct=True)
def to_label(p):
    if p > 0.8: return 4
    if p > 0.6: return 3
    if p > 0.4: return 2
    if p > 0.2: return 1
    return 0
```

This *cold-start* approach lets the model learn before any real user click data is collected. Once click logs accumulate, swap the pseudo-labels for the real ones and retrain.

Inference fallback. `ranking_ltr.py:LTRModel.load` never raises. If `lgbm_ltr.txt` is missing (which is the case in this checkout — the file is regenerated by `python manage.py train_ltr`), `predict()` returns zeros and the ensemble effectively becomes pure 70% heuristic. The user sees no error.

5.6.5 The five preset rankers

When NLU detects a strong ranking hint (e.g. “cheapest sugar”) the ensemble is bypassed in favour of a deterministic preset (`search_service.py:920-1042`):

Preset	Trigger	Sort key
<code>price_asc</code>	<code>price_asc</code> hint or “cheap”, “low price”	<code>avg_price</code> ASC (∞ if missing)
<code>price_desc</code>	<code>price_desc</code> hint	<code>avg_price</code> DESC
<code>volume_desc</code>	<code>volume_desc</code> hint or “biggest”, “largest”	<code>total_volume</code> DESC
<code>reliability</code>	<code>reliability</code> hint or “trusted”, “serious”	<code>shipment_count</code> DESC
<code>default</code>	none of the above	composite blended score

Table 5.5: Ranking presets.

The default composite (lines 992–1042) is:

$$0.4 \cdot v_n + 0.3 \cdot c_n + 0.2 \cdot \frac{1}{p_n} + 0.1 \cdot r_n$$

where each $\cdot_n$ is the min-max-normalised value of volume, count, inverse-price, and recency over the current batch.

5.7 Side quests: vector store, FAISS, OpenSearch

5.7.1 `vector_store.py`

`backend/search/services/vector_store.py` (128 lines) is a singleton wrapping a FAISS `IndexFlatL2` over `ProductSubCategory.name` embeddings. It is built lazily on first use and is the backing store for the legacy `CandidateRetriever`. The modern path (`SearchService`) does not use FAISS directly; it relies on `search_index.pkl` via `nlp.py`.

5.7.2 `nlp.py` (legacy `QueryMatcher`)

`backend/search/services/nlp.py` (156 lines) loads the pickled `search_index.pkl` into memory and runs three matching modes: keyword (exact), semantic (cosine on encoded query), and fuzzy variants via `difflib.SequenceMatcher` with cutoff 0.85. Used by older code paths that need the legacy semantic index.

5.7.3 OpenSearch (currently disabled)

The `sync_search` management command builds an OpenSearch index called `trade_index` with KNN enabled, but the live runtime path is gated:

```
# search_service.py:343
if False and self._opensearch_available() and subcat_ids:
    return self._opensearch_search(...)
```

The `if False` prefix is a feature flag the team flips during development. The ORM fallback handles all production traffic today.

5.8 Module-level singletons

Two singletons are created at module-import time:

- `global_search_service = SearchService()` at `search/views.py:16`. Loads the NLU engine, embedding model, and OpenSearch client *once per worker*.
- The NLU warmup thread (`search/apps.py`). Runs two throwaway `parse` calls so the heavy SetFit + GLiNER models load on boot, not on first request.

The result: cold start cost is paid before the worker starts serving traffic, and the per-request cost stays bounded.

5.9 Where the ML artefacts live

Artefact	Built by	Loaded by
<code>backend/models/zarai_intent_m</code>	<code>train_intent.py</code>	<code>nlu_engine.py</code> (SetFit body+head)
<code>backend/search_index.pkl</code>	<code>python manage.py build_index</code>	<code>nlp.py</code> (legacy matcher)
<code>backend/search/models/lgbm_lt</code>	<code>train_ltr.py</code>	<code>ranking_ltr.py:LTRModel</code>
<code>backend/*.graphml</code>	<code>build_gnn_graphs</code> command (in <code>companies</code> app)	Node2Vec / Similar Companies feature
<code>backend/search/models/intent_model/</code>		empty placeholder; the live model lives under <code>backend/models/</code>

Table 5.6: Search-related ML artefacts.

If the LightGBM file is missing, the system degrades to pure-heuristic ranking with no error. If the SetFit model is missing, the engine logs a warning and falls back to rule-based intent detection. If the search pickle is missing, the legacy matcher returns no semantic hits and the modern path’s PASS 0–5 cascade still works.

5.10 End-to-end trace, with code

To consolidate, here is the same query “*cheap basmati rice from Punjab to UAE*” traced through the search engine:

1. View `SearchViewSet.list` parses `q='cheap basmati rice from Punjab to UAE'`, computes `top_k=100`, calls `search_service.execute_search(...)`.

2. `execute_search` hits the NLU cache; on miss, runs `ModernNLUEngine.parse`. Result: `intent='BUY', product_keyword='basmati rice', country='United Arab Emirates', ranking_hint='pr`
3. `_resolve_subcategories` runs `PASS 0 ( ProductCategory.filter(name__icontains='basmati rice'))` which matches one category $\rightarrow$ subcategories `[42, 47, 53]`.
4. Scope-availability filter: `IMPORT` has plenty of rows for these subcats. No mismatch.
5. `_orm_search` delegates to `SupplierAggregator.get_suppliers_for_subcategories(...)`. Database returns 50 rows.
6. Ranking hint `price_asc` triggers the `price_asc` preset; the rows are re-sorted by `avg_price` ascending.
7. View applies the paywall preview slicing (max 2 visible if user has no entitlement). Returns JSON.
8. Frontend `SearchResults.js` renders the supplier list, each row clickable into `DealDetail.js`.

5.11 The 9 known gaps (be honest with yourself)

The exploration of the search code surfaced the following ground-truth notes you should know:

1. `backend/search_index.pkl` is *not committed*; you must build it via `python manage.py build_search_index` before the legacy matcher works.
2. `backend/search/models/lgbm_ltr.txt` is *not committed*; the LTR model degrades to zero output, so the ensemble runs as pure heuristic until you train it via `python manage.py train_ltr` (or `python -m search.services.train_ltr`).
3. `ltr_evaluation.py` source is missing in this checkout; only the compiled `.pyc` survives. The trainer references it but the import is wrapped in a try/except.
4. OpenSearch path is hard-disabled by `if False and ...` at `search_service.py:343`. The team kept the indexer in tree as a migration target.
5. The live SetFit body is `paraphrase-MiniLM-L6-v2`, despite `train_intent.py` declaring `BAAI/bge-small-en-v1.5`. The trainer can be re-run to produce the bge model; the runtime loader picks up whatever is on disk.
6. The live engine uses GLiNER (not KeyBERT) for product extraction. KeyBERT is loaded but currently unused.
7. Families 6, 7, 8 are defined in `query_parser.py` but absent from the `FAMILY_WEIGHTS` table; family 9 (Hybrid) is the safety net.
8. `backend/search/models/intent_model/` is an empty placeholder; the SetFit trainer writes there but the runtime loader reads `backend/models/zarai_intent_model/`. A manual move/rename is implied at deploy time.
9. The `scope_match` feature is constant 1.0 because retrieval already enforces scope; the slot is reserved for a future relaxed-retrieval mode.

Chapter 6

Frontend Deep Dive

This chapter walks through the entire React application living under `frontend/`. It is a Create React App (CRA) project written in modern functional React with hooks, Tailwind CSS for utility styling, and custom CSS modules for the design system. We cover configuration, the provider/route tree, every shared building block, and every page.

6.1 Tooling and configuration

6.1.1 `frontend/package.json`

The CRA scaffold pinning React 19, Tailwind, recharts, framer-motion, and the icons set. The interesting non-trivial dependencies:

Package	Use in Zarailink
<code>axios</code>	Single HTTP client (<code>services/api.js</code>) for every backend call.
<code>react-router-dom</code>	SPA routing, <code><Router></code> + <code><Routes></code> live in <code>App.js</code> .
<code>recharts</code>	The active chart library for sparklines, line/bar/pie.
<code>framer-motion</code>	Page transitions, dropdown animations, floating orbs on the dashboard.
<code>lucide-react</code>	Icon set used throughout (Search, Lock, ArrowRight, etc.).
<code>react-simple-maps</code>	World map in <code>CompanyOverview.js</code> .
<code>shepherd.js</code>	Onboarding tour the dashboard runs the first time you log in.
<code>jspdf + jspdf-autotable</code>	Client-side PDF export in <code>ComparePage.js</code> and <code>DealDetail.js</code> .
<code>react-hot-toast</code>	Toast notifications via <code>ToastProvider.js</code> + <code>useToast.js</code> .
<code>web-vitals</code>	Performance metrics; declared but the report callback is not wired.

Table 6.1: Frontend dependencies of note.

Scripts. `start` (CRA dev server), `build` (production bundle into `frontend/build/`), `test` (Jest watch mode), `eject` (one-way escape).

Dormant deps. `chart.js` and `react-chartjs-2` are listed but unused; `html2canvas` is declared but never invoked; `driver.js` is declared but the actual tour uses Shepherd.

6.1.2 `tailwind.config.js` and `postcss.config.js`

```
module.exports = {
  content: ["../src/**/*.{js,jsx,ts,tsx}"],
  theme:    { extend: {} },
  plugins: [],
};
```

Listing 6.1: `tailwind.config.js`

The config is intentionally minimal. The team did not customise the theme in Tailwind itself; instead the design system lives in `src/index.css` as CSS variables, which Tailwind utilities reference where needed.

6.1.3 `frontend/.env`, `.env.local`, `.env.production`

The key variable everywhere is `REACT_APP_API_BASE_URL`:

- `.env`: `http://localhost:8000` (dev default).
- `.env.local`: same (overrides per-developer).
- `.env.production`: `https://whinier-winter-ruttily.ngrok-free.dev` (the team’s persistent ngrok demo URL).

This is the only env variable; everything else is hardcoded.

6.1.4 `public/ assets`

`public/index.html` is the CRA HTML shell with two extra `<link rel="stylesheet">` tags loading Google Fonts: *DM Sans* and *Inter*. `public/manifest.json` is the default Create-React-App manifest — `name` and `short_name` were never customised (they read “*React App*”). Worth fixing as a five-minute polish task.

6.1.5 `src/index.js` and `src/index.css`

`src/index.js` is 17 lines: `ReactDOM.createRoot` mounts `<App/>` inside `<React.StrictMode>`, imports the global CSS, and calls `reportWebVitals()` (no callback, so metrics aren’t actually reported anywhere).

`src/index.css` is 330 lines and is the heart of the design system:

- Imports *Inter* from Google Fonts.
- Three Tailwind directives: `@tailwind base/components/utilities`.
- A complete colour palette as CSS variables on `:root` — emerald primary (`\#10b981`), slate backgrounds, four shadow tiers.
- A `:root.dark` override block for dark mode (emerald `\#34d399`, slate-900 backgrounds).

- Global `* rule` that animates background/border/colour transitions everywhere.
- A `.radar-container` loader animation (conic gradient rotating 360, with concentric pulse rings) used while a search runs (`SearchResults.js:686-696`).
- Custom Shepherd.js theme classes for the onboarding tour.

Warning

The dark-mode classes exist but no code currently sets the `dark` class on `<html>`. `ThemeContext.js` toggles a boolean, but only `ToastProvider.js` reads it. So dark mode is dormant — turning it on requires wiring the boolean into a `document.documentElement.classList.toggle('dark')` effect.

6.2 The provider and route tree (App.js)

`frontend/src/App.js` (234 lines) is the application skeleton. The provider stack is:

```
<ThemeProvider>
  <ToastProvider>
    <AuthProvider>
      <Router>
        <Suspense fallback={<PageLoader/>}>
          <Routes> ... </Routes>
        </Suspense>
      </Router>
    </AuthProvider>
  </ToastProvider>
</ThemeProvider>
```

Listing 6.2: Provider tree (App.js:79–84)

Helper components.

- `PageLoader` (lines 34–38) — centred spinning ring shown while a lazy chunk loads.
- `ProtectedRoute` (lines 41–57) — pulls `isAuthenticated`, `loading` from `useAuth()`. Renders `children` when authenticated; redirects to `/login` otherwise.
- `PublicRoute` (lines 60–76) — mirror image; redirects *authenticated* users away from `/`, `/login`, `/signup` to `/dashboard`.

The route table.

Warning

Notice that `/dashboard`, `/search/results`, `/search/supplier/:name`, and `/search/compare` are *not* wrapped in `ProtectedRoute`. They are reachable without login — but features inside them (paywalls, deal detail drill-downs, etc.) check `isAuthenticated` themselves. This gives an unauthenticated visitor a teaser experience.

Path	Wrapper	Component
/	PublicRoute	LandingPage
/login	PublicRoute	Login
/signup	PublicRoute	Signup
/forgot-password	none	ForgotPassword
/reset-password/:token	none	ResetPassword
/dashboard	none (open)	Dashboard
/profile	ProtectedRoute	UserProfile
/trade-directory/find-suppliers	ProtectedRoute	FindSuppliers
/trade-directory/find-buyers	ProtectedRoute	FindBuyers
/trade-directory/company/:id	ProtectedRoute	CompanyProfile
/watchlist	ProtectedRoute	Watchlist
/trade-intelligence/link-prediction	ProtectedRoute	LinkPrediction
/trade-intelligence/company/:id	ProtectedRoute	CompanyOverview
/trade-intelligence/company/:id	ProtectedRoute	CompanyProducts
/trade-intelligence/company/:id	ProtectedRoute	CompanyPartners
/search	none	Navigate to /dashboard
/search/results	none (open)	SearchResults
/search/supplier/:name	none (open)	DealDetail
/search/compare	none (open)	ComparePage
/subscription	ProtectedRoute	Subscription
*	none	Navigate to /

Table 6.2: Routes defined in App.js.

Lazy vs eager. Heavy pages (the four Trade Intelligence pages, all four Search pages, UserProfile) are `React.lazy()` so the initial bundle stays small. Auth pages, the Dashboard, FindSuppliers/FindBuyers, CompanyProfile, Subscription, and Watchlist are eager.

6.3 HTTP layer (services/)

6.3.1 services/api.js

11 lines:

```
import axios from 'axios';
const api = axios.create({
  baseURL: `${process.env.REACT_APP_API_BASE_URL}/api`,
  withCredentials: true,
  headers: { 'Content-Type': 'application/json' },
});
export default api;
```

Listing 6.3: services/api.js

The three properties to remember:

1. `baseURL` prepends `/api` to every call — so `api.get('/search/')` hits `${REACT_APP_API_BASE_URL}/api/search/`

2. `withCredentials: true` sends the session cookie cross-origin — mandatory for Django session auth.
3. Default JSON content type.

6.3.2 `services/searchService.js`

73 lines, exports an object with four methods:

Method	Endpoint	Params
<code>search(query, filters)</code>	GET <code>/api/search/</code>	<code>q</code> + filters
<code>getSupplierDetails(...)</code>	GET <code>/api/search/supplier-d</code>	<code>name</code> , <code>query</code> , <code>scope?</code> , <code>subcat_id?</code> , <code>variant_name?</code> ,
<code>compareSuppliers(...)</code>	GET <code>/api/search/compare/</code>	<code>suppliers</code> (csv), <code>query</code>
<code>getSupplierTransactions</code>	GET <code>/api/search/supplier-t</code>	<code>page</code> , <code>size</code> , <code>filters</code>

Table 6.3: `searchService.js` methods.

6.4 Contexts

6.4.1 `AuthContext.js` (91 lines)

State: `user` (null until logged in), `loading` (true until the initial `checkAuthStatus` resolves).

Effect: on mount, calls `GET /accounts/api/check-auth/`; on success sets `user` to the returned user object.

API exposed via `useAuth()` :

- `user`, `isAuthenticated`, `loading`.
- `login(email, password)` — `POST /accounts/api/login/`, on success sets `user` state.
- `logout()` — `POST /accounts/api/logout/`, clears state.
- `checkAuthStatus()` — the bootstrap call (also called manually after token spend).
- `refreshUser()` — re-fetch `check-auth` without redirecting; used to refresh the token balance after an unlock or purchase.

6.4.2 `ThemeContext.js` (16 lines)

Tiny: holds a boolean `darkMode` in state and exposes a `toggleTheme()` setter. Currently no effect side-loads the boolean into the DOM, so the dark CSS variables defined in `index.css` are inactive — but the API is in place for when someone ships dark mode.

6.5 Hooks

`frontend/src/hooks/` contains five custom hooks.

- `useDebounce(value, delay)` (48 lines) — standard debounced state. Plus `useDebouncedCallback(fn, delay)` variant for callbacks.
- `useToast()` (165 lines) — thin wrapper over `react-hot-toast` exposing eight helpers: `success`, `error`, `loading`, `custom`, `info`, `warning`, `promise`, `dismiss`. Reads `darkMode` from `ThemeContext` so toasts auto-style.
- `useWatchlist()` (66 lines) — localStorage-backed watchlist of companies. Exposes `watchlist`, `addToWatchlist`, `removeFromWatchlist`.
- `hooks/index.js` (10 lines) — re-exports five hooks (`useFilterPersistence`, `useActivityHistory`, `useCompanyNotes`, `useDebounce`, `useDebounceCallback`) whose source files do not exist in the repo.

Warning

`CompanyNotes.js` imports `useCompanyNotes` from `hooks/index.js`, but the source file is missing. The component will throw at module-load if anything renders it.

6.6 Shared components (`components/Common/`)

The shared building-blocks. Many have a `*.css` sibling for custom styling.

Component	Purpose
<code>Pagination.js</code>	Numbered + previous/next paginator. Used in directory pages and disambiguation lists.
<code>Modal.js</code>	Reusable modal with overlay + close-on-Escape.
<code>EmptyState.js</code>	Centred icon + message + optional CTA for empty grids.
<code>Skeleton.js</code>	Animated pulse-loading rectangles.
<code>ExportButton.js</code>	Drop-down with PDF/CSV/Excel options; the actual export logic is a stub in <code>exportUtils.js</code> .
<code>ShareButton.js</code>	Copy-link or native share UI.
<code>WatchlistButton.js</code>	Wraps <code>useWatchlist()</code> into a star button used on every company card.
<code>VerificationBadge.js</code>	Pill rendering verification status (Verified / Pending / Premium).
<code>SortSelector.js</code>	Native <code><select></code> with custom options array.
<code>Breadcrumb.js</code>	Slash-separated breadcrumb.
<code>Charts.js</code>	Re-exports four recharts pre-styled chart wrappers used in DataDashboard and DealDetail.
<code>ToastProvider.js</code>	Mounts <code><Toaster/></code> from <code>react-hot-toast</code> , theme-aware.

Table 6.4: Shared components.

6.7 Layout

`components/Layout/Navbar.js` (176 lines) is the persistent top navigation. It shows the brand, nav items (Dashboard, Trade Directory, Trade Intelligence, Watchlist, Subscription), and the

user menu (avatar, token balance pill, profile link, logout). Uses `framer-motion` to animate the dropdown.

6.8 Marketing

`components/Marketing/LandingPage.js` (330 lines) is the unauthenticated home page. Hero section with headline, three feature columns, CTA to `/signup`.

6.9 Auth pages

- `Auth/Signup.js` (261 lines) — form with name, email, password, country. Calls `POST /accounts/api/signup` and on success calls `login(email, password)` to immediately log the new user in.
- `Auth/Login.js` (161 lines) — email + password form. On success navigates to `/dashboard`.
- `Auth/ForgotPassword.js` (163 lines) — email-only form. Currently mocked: it never calls a backend endpoint, just shows a success toast after a timeout. Wire it to `POST /accounts/api/forgot-password` when needed.
- `Auth/ResetPassword.js` (240 lines) — two-password form. Also mocked.

6.10 Dashboard (`components/Dashboard/Dashboard.js`, 726 lines)

The post-login home. Despite its name it is largely a search landing page; the analytics dashboard is at `/search/results?mode=dashboard` (see `DataDashboard`).

Inner components.

- `AnimatedCounter` — `IntersectionObserver`-based counter that animates from 0 to a target value on scroll-in using `requestAnimationFrame` with an ease-out cubic.
- `FloatingOrb` — a `framer-motion` div animating *y* and scale on a 6-second sinusoid forever.

Effects.

1. *Onboarding tour*: if `localStorage` key `zarailink_tour_done` is missing, after a 1-second delay it builds a `Shepherd.Tour` with up to seven steps: search bar, four search modes, scope toggle, AI query example, example queries, nav Intelligence, nav Subscription. The tour writes `true` to `localStorage` on `complete` or `cancel`.
2. *Autocomplete fetch*: 250 ms debounce; numeric queries hit `/api/search/hs-code-tree?q=`, alpha hits `/api/search/autocomplete?q=`.
3. *Click-outside handler* for the suggestions dropdown.

Layout summary.

- Hero section with three `FloatingOrbs` (emerald, teal, blue).

- “4 Ways to Search ZaraiLink” card with sub-cards for HS Code, Category, Product Name, AI Query.
- Big headline “What are you looking for, {firstName}?”.
- Search bar (id `tour-search-bar`).
- Floating mode badge (HS Code / Product / AI).
- Autocomplete dropdown (`AnimatePresence`).
- Scope toggle (Imports / Exports).
- Intent pills (“I want to buy”, “I want to sell”, etc.).
- Example queries.
- Stats bar (slate-900) with three `AnimatedCounters`: 50k+ records, 2k+ companies, 500+ products.
- Features grid: four cards (Discover, Understand, Profiles, Contacts).
- CTA card with two CTAs to Browse Suppliers / Buyers.

6.11 Search pages

The most complex part of the frontend.

6.11.1 SearchHome.js (333 lines)

A near-duplicate of the search section of Dashboard. With the App.js routing change, `/search` now redirects to `/dashboard`, making this component effectively unreachable.

6.11.2 SearchResults.js (919 lines)

The router/dispatcher behind `/search/results`. Reads roughly 30 URL parameters and 30 React state pieces, then *forks* the render tree:

- If `isRawHsCodeMode` (numeric query) and the digits are 7+ or `mode=dashboard` → render `<DataDashboard/>`.
- If `isRawHsCodeMode` but fewer than 7 digits → render `<SummaryView/>`.
- Otherwise render the standard ranked-results UI.

Standard results UI.

- Sticky search bar at the top.
- Left filter sidebar: Country, Price Range Min/Max, Min Volume.
- Main column: scope-mismatch banner (“no IMPORT data — try EXPORT”), broad-search warning, disambiguation panel (12 per page), results header with four trade-direction pills, sort selector, radar loader during load, supplier cards, paywall CTA.
- Right column ($\geq$ lg breakpoint): market snapshot card showing avg price, top origin/destination, active search engine.
- Sticky bottom Compare bar when 1–4 suppliers are selected, linking to `/search/compare`.

Paywall. If `accessState !== 'FULL_ACCESS' && accessState !== 'PRODUCT_ACCESS'`, only the first two suppliers are clickable; the rest are blurred with an unlock CTA. Clicking the CTA

calls `handlePurchase()` which POSTs to `/api/subscriptions/purchase-access/`.

6.11.3 SummaryView.js (133 lines)

Lightweight HS-code summary list. Renders rows of subcategory cards from `GET /api/search/hs-summary/`. Used when the user navigates to a non-leaf HS code.

6.11.4 DataDashboard.js (634 lines)

The analytics view shown for 7-digit HS codes. Built around *four pills* (`FOREIGN_SUPPLIERS`, `FOREIGN_BUYERS`, `PAYMENT_METHODS`, `PRODUCTS`) which map to (scope, intent) pairs.

Layout.

- Slate-900 sticky header: “Market Intelligence” eyebrow, HS code, description, total shipments badge, active filter pill, Dashboard back button.
- Left sidebar: “Refine by Product” search + checkbox list with per-subcategory shipment counts, Clear All button.
- 4-pill switcher for trade direction.
- Profile cards: name, country, four metrics (Avg Price, Volume, Shipments, Avg Shipment), last shipment date.
- Inline paywall teaser if `accessState == 'NO_ACCESS'` — token cost is $500 \times$ refinements if any are selected, otherwise 5,000 for the whole HS code.
- Sticky compare bar.

6.11.5 ComparePage.js (456 lines)

`/search/compare`. Side-by-side table of up to four suppliers. Pulls data via `searchService.compareSuppliers`.

Winner highlighting. `getWinners()` finds the best cell per metric (min price, max volume, max shipments, most recent). Best cell gets an emerald background and a CheckCircle badge.

PDF export. `exportPDF()` (165 lines) builds a landscape A4 jsPDF document with: header bar (green “ZaraiLink”), metadata, the comparison table via `autoTable`, per-supplier “Historical Price Trends” tables (price points filtered to the 0–10,000 USD/MT range), and a footer with page numbers. Saved as `ZaraiLink_Comparison_<variant>_<date>.pdf`.

6.11.6 DealDetail.js (1,688 lines)

The supplier/buyer drill-down at `/search/supplier/:name`. The single largest file in the frontend. Four sub-tabs:

1. **OverviewTab.** Renders the `aggregator._compute_overview_metrics` payload: active period, typical shipment size, geographic data, top counterparties, shipment-size distribution, frequency pattern, behavioural summary, market position labels.
2. **MarketPricingTab.** Renders the `_compute_market_pricing` payload: market overview, monthly price trend chart, positioning vs market, country pricing, supply-chain flow (top 10 routes), demand trends, competitive benchmarking (top 50 competitors).

3. **CompanyTab.** Renders `_compute_company_profile`: portfolio (top 5 products), network (top partners + concentration), geography, trends (quarterly via `TruncQuarter`, YoY growth), behaviour (unique partners, repeat ratio, long-term partners), per-product top-3 partners.
4. **TransactionsTab.** Server-side filtered+paginated transactions via `searchService.getSupplierTransactions`. Filters: date range, buyer, seller, country, qty range, price range. 15 rows per page.

A `WatchlistButton` lives in the header, plus a Compare-with-others CTA, an Unlock-Contacts CTA, and a Download-PDF action that mirrors the `ComparePage.js` export.

6.12 Trade Directory pages

6.12.1 FindSuppliers.js (310 lines)

Effects.

1. On mount: parallel `fetch /api/sectors/` and `/api/company-roles/`; finds the role named `supplier` or `suppliers`, captures its id, then `fetch /api/companies/countries/?role=...` to populate the country drop-down.
2. `fetchCompanies` (memoised) — whenever role, debounced search, country, or sector change, calls `GET /api/companies/?role=...&search=...&country=...§or=...`.

UI. Hero header with title and switch-to-buyers button; search bar with clear button; collapsible filter panel (Country, Sector, Sort, Contact Status); result grid of company cards with avatar, name, chips for country/sector/type/Verified Contacts, and a black CTA linking to `/trade-directory/company/<id>`; bottom Pagination. Loading state uses skeleton placeholders.

6.12.2 FindBuyers.js (310 lines)

Mirror of `FindSuppliers`, but blue accent colour and looks up role `buyer` / `buyers`.

6.12.3 CompanyProfile.js (449 lines)

`/trade-directory/company/:id`. Composes:

- Header: back button, avatar, name, optional Verified pill, location summary.
- About section, quick stats grid (Sector, Role, Type, Established, Employees, Contacts).
- Optional website link and email.
- Products section — pill list of `CompanyProduct` rows, optional Value Added amber chips.
- Key Contacts hero section with subtitle showing unlock progress and the user's token balance.
- Grid of `<ContactCard>` components.
- Low-balance amber banner if user has 0 tokens, with link to `/subscription`.
- Mounted modals: `UnlockConfirmModal`, `SuccessModal`, `InsufficientTokensModal`, `ErrorModal`.

Unlock flow.

1. Click on a contact card → opens `UnlockConfirmModal` .
2. Confirm → `POST /api/key-contacts/<id>/unlock/` with the CSRF token from `document.cookie` .
3. On success: `refreshUser()` , store contact info, show `SuccessModal` , refetch company data after 500 ms.
4. On 402: `InsufficientTokensModal` .
5. On other errors: `ErrorModal` .

Helpers. `getSeniorityBadge(designation)` keyword-sniffs into Decision Maker / Procurement / Sales / Management / Team Member chips. `maskValue(val)` returns placeholder strings like `fâĂcăĂcăĂcăĂcă@gmail.com` or `+92 âĂcăĂcăĂcăĂcă-âĂcăĂcăĂcăĂcăĂcăĂcăĂcă` .

6.12.4 CompanyNotes.js (67 lines)

A small note-taking card for company pages. Imports `useCompanyNotes` from `hooks/index.js` but the source file is missing — will throw at import. Treat as a half-finished feature.

6.13 Trade Intelligence pages

Four pages under `/trade-intelligence/` :

- `CompanyOverview.js` (452 lines) — header, tab switcher, metric cards (Total Volume, Estimated Revenue, Top 5 Products, Top 5 Partners), Partner Geography (a Mercator world map via `react-simple-maps` with marker circles sized by volume) and Volume Trend (pure-SVG line chart), filter analysis. Calls `GET /api/company/<id>/overview/` .
- `CompanyPartners.js` (376 lines) — sortable Top Partners table, Partner-Country bars, Monthly Trade Activity recharts `LineChart` (one line per partner), Key Products Traded with Partners. Calls `GET /api/company/<name>/partners/` .
- `CompanyProducts.js` (362 lines) — Product Portfolio table, recharts donut `PieChart` of top 5 categories, average price recharts `LineChart`, top suppliers/buyers by product, primary partner per product. Calls `GET /api/company/<name>/products/` .
- `LinkPrediction.js` (301 lines) — predicts likely sellers/buyers for a company. Drop-downs for prediction type + method, autocompletes company names from `/api/explorer/` , calls `/api/predict/<sellers|buyers>/<name>/?method=&top_k=10` , renders ranked card grid with Confidence %, Tier badge, and per-method score breakdown.

6.14 Watchlist

`components/Watchlist/Watchlist.js` (236 lines). Reads the client-side watchlist from `useWatchlist()` ; supports search filter, sort selector (Recently Added / Oldest / Name A-Z / Z-A), Clear All with confirm. Each card has a remove × button. Empty state links to `FindSuppliers` / `FindBuyers`.

6.15 Subscriptions

`components/Subscriptions/Subscription.js` (190 lines). Pulls plans from `GET /api/subscriptions/plans/`, filters by billing cycle (annual / monthly) client-side, and lets the user click “Get Plan”. The handler does `fetch POST /api/subscriptions/redeem/` with `X-CSRFToken` header and `\{plan_id\}` body. On success it alerts the credited token count and calls `refreshUser()`.

6.16 User profile

`components/Profile/UserProfile.js` (389 lines). Three tabs:

- **Account** — name + email change, password change.
- **Entitlements** — list from `GET /api/subscriptions/profile-data/`. Shows what HS-code / product accesses the user owns.
- **Token History** — the same endpoint’s `tokenHistory` array, sorted descending.
- **Telemetry** — the queries / productsViewed / tokensSpent / topCategory derived metrics.

6.17 Cross-cutting concerns

6.17.1 Token economy on the frontend

Every place where the user can spend tokens routes through one of these two patterns:

1. **Direct unlock** (key contacts): `POST /api/key-contacts/<id>/unlock/`, charges 1 token. Used in `CompanyProfile.js`.
2. **Access purchase** (search results): `POST /api/subscriptions/purchase-access/`, charges 500 (single product) or 5,000 (full HS code). Used in `SearchResults.js` and `DataDashboard.js`.

After both, the frontend calls `refreshUser()` so the navbar balance pill stays current.

6.17.2 Session-cookie auth model

Every `api.get/post` carries the `sessionid` cookie because of `withCredentials: true`. Some POSTs (notably the unlock and the redeem endpoints) also include an `X-CSRFToken` header read from `document.cookie` via the `getCookie('csrftoken')` helper, because Django enforces CSRF on session-authenticated POSTs.

6.17.3 The SearchResults dispatcher

A single component handles three render trees by URL inspection. This is what gives the user the “one search bar, three experiences” feel without any client-side state machine.

6.18 Frontend tests

Each shared component plus the hooks have Jest + React Testing Library coverage:

- `Common/__tests__` : Pagination, EmptyState, VerificationBadge, WatchlistButton.
- `hooks/__tests__` : `useDebounce`, `useToast`, `useWatchlist`.
- `context/__tests__/ThemeContext.test.js` .

Run them with:

```
cd frontend && npm test
```

6.19 Open issues to be aware of

1. `hooks/index.js` re-exports five hooks (`useFilterPersistence`, `useActivityHistory`, `useCompanyNotes`, `useDealDetail`, `useWatchlist`) whose source files do not exist. `CompanyNotes.js` will crash if rendered.
2. Dark mode CSS exists but no DOM toggle wires it on; only `ToastProvider.js` reads the boolean.
3. `ForgotPassword.js` and `ResetPassword.js` are mocked — no API calls fire.
4. `utils/exportUtils.js` is a stub (`alert(...)`); only the inline jsPDF code in `ComparePage` / `DealDetail` actually exports.
5. `/dashboard`, `/search/results`, `/search/supplier/:name`, `/search/compare` are open routes.
6. `/trade-intelligence/ledger` is referenced as a back-link target from `CompanyOverview` / `Partners` / `Products` but no route is defined.
7. `chart.js` and `react-chartjs-2` are installed but unused. `recharts` is the only active charting library.

Chapter 7

The Data Layer

This chapter is the field guide to every data file shipped in the repository: spreadsheets, JSON fixtures, GraphML graphs, ML model artefacts, and the scripts that move data between them.

7.1 Source spreadsheets

File	Bytes	Contents
<code>companies.xlsx</code>	20,965	35 Pakistani sugar-mill suppliers across 14 columns (role, type, name, country, founded, employees, website, email, phone, address, description, sector, created_at, updated_at). Sheet 'Association 1' is the data; 'Sheet1' is empty placeholder.
<code>keycontacts.xlsx</code>	10,703	4 key-contact rows: Company, Name, Designation, What-sapp, Email.
<code>import_data_1year.xlsx</code>	334,491	2,232 IMPORT transactions covering ~12 months ending 2024-11. Header is on row 6 of Sheet1. 15 columns.
<code>export_data.xlsx</code>	665,646	7,196 EXPORT transactions, same 15 columns but Buyer/Seller are swapped.
<code>backend/companies.csv</code>	6,642	44 supplemental enrichment rows (manufacturer types, descriptions, websites). Joined onto existing Company rows by name.

Table 7.1: Source spreadsheets and CSV.

7.1.1 Inside `import_data_1year.xlsx`

15 columns: Date, HS Code, Category, Sub-Category, Item Description, Buyer, Seller, Shipping Agents, Sample rows (truncated):

- 2024-11-01 | 1702.3 | Glucose and glucose syrup | Dextrose Anhydrous | "Pharmaceutical raw materials"
- 2024-11-01 | 1704.909 | Other | Jelly (Assorted Flavor)
- 2024-11-04 | 1702.902 | Caramel | Caramel Color

7.1.2 Inside `export_data.xlsx`

Same 15 columns, but Buyer/Seller are reversed: now *Pakistan is the seller*, the foreign side is the Buyer. The ingestion pipeline detects this by file name and uses the `ingest_trade_export` command, which sets `origin_country = "Pakistan"` and `destination_country = row.country`.

Worked Example

Sample export row: 2024-11-01 | 1701.991 | White crystalline cane sugar | Refined Sugar | "SUGAR (

7.2 JSON fixtures

Django *fixtures* are JSON arrays of `\{model, pk, fields\}` objects produced by `python manage.py dumpdata` and consumed by `python manage.py loaddata`. The repo ships several:

File	Objects	Notes
<code>load_data.json</code> (root)	18,613	Full fixture: 10k TradeLensTransactions, 3k ProductItems, 2.2k Transactions, 1.5k ProductEmbeddings, 700 CompanyEmbeddings, 580 ProductSubCategories, etc.
<code>load_data_clean.json</code> (root)	18,574	Same minus 39 <code>contenttypes.contenttype</code> rows that conflict with auto-created types on a fresh DB. Generated by <code>load.py</code> .
<code>backend/load_data.json</code>	10,390	Includes 1,846 <code>auditlog.logentry</code> rows but omits TradeLens.
<code>backend/load_data_clean.json</code>	193	Auth + subscription slice only.
<code>backend/load_data_utf8.json</code>	1,206	UTF-8 re-encoding focused on the older <code>trade_ledger</code> schema.
<code>backend/subscriptions_only.json</code>	73	Just plans + redeem codes.
<code>backend/schema_dump.json</code>	22 specs	A schema description (not a fixture). Source of <code>ZARAILINK_DATA_DESIGN.md</code> .
<code>backend/audit_broad.json</code>	121 rows	Search-debugging dump of <code>product_item</code> counts.
<code>backend/audit_seawall.json</code>	1 supp.	Per-supplier audit (Seawall Enterprise Ltd, 37 IM-PORT tx).

Table 7.2: JSON fixtures and dumps.

To seed an empty DB:

```
python manage.py loaddata ../load_data_clean.json # full demo
# or just the subscription plans:
python manage.py loaddata subscriptions_only.json
```

7.3 GraphML files

Seven NetworkX-format graph files live in `backend/`. They are the inputs to the Node2Vec embedding pipeline that powers the *Similar Companies* feature and the *Link Prediction* experi-

ments.

File	Nodes	Edges	Semantic role
buyer_seller_graph.graphml	710	717	Bipartite buyer↔seller co-occurrence.
company_company_graph.graphml	710	717	Generic company-to-company co-trade.
company_country_graph.graphml	449	583	Company↔country bipartite.
company_product_graph.graphml	711	1,836	Company↔product bipartite. Drives “what does this company sell”.
product_co_trade_graph.graphml	681	13,552	Product-to-product co-occurrence. Used for “Similar Products”.
product_country_graph.graphml	760	1,750	Product↔country bipartite.
seller_product_graph.graphml	701	1,812	Seller-only company↔product subset.

Table 7.3: GraphML files and their semantics.

How they are produced. `python manage.py build_gnn_graphs` (in the `companies` app) walks every `Transaction` and emits the seven files. The setup script triggers this only if the graphs are older than the source XLSX or missing entirely.

How they are consumed. `python manage.py generate_gnn_embeddings -fast` runs `Node2Vec` random walks over each graph and stores 128-dim embeddings in `trade_data.CompanyEmbedding.embedding` (a `JSONField`). The “Similar Companies” feature, the `LinkPrediction.js` page, and the recommendation tab on `DealDetail.js` all consume those vectors.

7.4 ML model artefacts

Three runtime artefacts. Two are *not committed* to the repo (too big or train-on-demand) and must be regenerated locally:

Artefact	Status	Regenerate via
backend/models/zarai_intent_model/ (Git LFS)	(Git LFS)	<code>train_intent.py</code>
backend/search_index.pkl	not committed	<code>python manage.py build_search_index</code>
backend/search/models/lgbm_ltr.pkl	not committed	<code>python manage.py train_ltr</code> or <code>python -m search.services.train_ltr</code>
backend/checkpoints/checkpoint-intro700 (LFS)	(Git LFS)	checkpoint of the SetFit training run

Table 7.4: ML artefacts.

7.4.1 The intent model

`backend/models/zarai_intent_model/` is a ~87 MB SetFit model on top of `paraphrase-MiniLM-L6-v2` (BERT, 6 layers, 384-d embeddings). Files inside:

- `model.safetensors` (90.8 MB) — frozen sentence-BERT encoder weights.

- `tokenizer.json`, `vocab.txt`, `tokenizer_config.json`, `special_tokens_map.json` — HF tokenizer.
- `config.json`, `config_sentence_transformers.json`, `sentence_bert_config.json`, `1_Pooling/config.json` — model and pooling config (max sequence 128, mean-pooling).
- `config_setfit.json` — the binary intent labels: `\{normalize_embeddings: false, labels: ["BUY", "S`
- `model_head.pkl` — pickled scikit-learn `LogisticRegression` head.
- `modules.json` — the module pipeline (Transformer → Pooling).

Reported accuracy from `Data_NLU_Pipeline_Details.md`: 89.4% on a held-out set of 354 trade queries.

7.4.2 The search index pickle

A single pickled dict produced by `python manage.py build_search_index`: `\{ids:[], names:[], hs_codes:[],`
Loaded once at boot by `search/services/nlp.py`; used by the legacy QueryMatcher’s semantic-similarity path.

7.4.3 The LightGBM LTR model

A plain-text LightGBM Booster dump (`backend/search/models/lgbm_ltr.txt`). Trained by `train_ltr.py`. If missing, the runtime degrades to *pure heuristic* ranking with no error.

7.5 Ingestion scripts

7.5.1 The two load_data.pys

- `load_data.py` (root, 10 KB) — legacy entry point with *Windows-hardcoded paths*. Bootstraps Django, calls `import_companies()` and `import_contacts()`.
- `backend/load_data.py` — portable twin (uses `Path(__file__).resolve().parent`). Production-ready.
- `backend/load_data copy.py` — developer’s literal backup of the root version with the Windows path.

7.5.2 Excel ingestion commands

Already covered in §4.5. The cheat-sheet:

```
python manage.py ingest_trade          --file ../import_data_1year.xlsx  # ↩
IMPORTs
python manage.py ingest_trade_export  --file ../export_data.xlsx      # ↩
EXPORTs
python manage.py repair_trade_companies  # ↩
backfill Company table
```

7.5.3 Companies enrichment

- `backend/import_companies.py` — updates existing `Company` rows from `companies.csv`, using a `MANUAL_MAP` dict to resolve fuzzy mismatches (`'meggle group' $ \to $ 'meggle gmbh \& co kg'`, etc.).
- `backend/match_companies.py` — read-only diagnostic sibling: reports which CSV rows have an exact, partial, or no DB match. No writes.

7.5.4 Search-index commands

- `python manage.py build_search_index` — builds the pickle.
- `python manage.py sync_search [-reset]` — builds the OpenSearch `trade_index` (gated off in the runtime).

7.5.5 Smart-search vector index

`python manage.py index_companies` populates a Redis HNSW vector index (`idx:companies`) keyed by company id with 1536-d OpenAI embeddings. Used when the user clicks “Use AI search” on the directory page.

7.6 Diagnostic and dev scripts

7.6.1 In backend/ root

7.6.2 In repo root

- `check_excel.py` — pandas one-shot probe of an Excel.
- `load.py` — 14-line script that strips `contenttypes.contenttype` rows from the fixtures.
- `strip_comments.py` — walks `backend/` and `frontend/src/` stripping comments via `tokenize` (Python) and a string/regex-aware FSM (JS).
- `test_api.py`, `test_endpoints.py`, `test_mapped.py`, `test_real_security.py` — smoke and security probes against the running server.

7.7 Visual assets

- `nlu_pipeline.svg` (5.5 KB) — 800×650 diagram of the 6-step NLU pipeline. Includes the gold dashed bypass arrow showing that the DeepSeek price-extraction step is skipped when no price signal is detected.
- `system_architecture.svg` (7.5 KB) — 800×500 three-layer architecture diagram (React UI → Django REST → NLU + PostgreSQL).
- `zarailink_fyp_poster.html` (16 KB) — A0 portrait FYP poster in HTML/CSS, print-ready.
- `backend/zarailink_erd.png` (220 KB) — the entity relationship diagram for the entire database schema.

Script	Purpose
<code>diagnose_typo_queries.py</code>	5-stage NLU tracer for failing/working queries; output committed at <code>diagnose_output.txt</code> .
<code>rediagnose.py</code>	Re-runs the diagnostic after the “Bug 1 fix” (preposition leak). Adds Brazil/UAE inspection.
<code>perf_test.py</code>	Spawns the dev server, hits a curated endpoint set 3× each, asserts SLA targets (500 ms autocomplete, 3 s otherwise).
<code>reliability_test.py</code>	Negative-input smoke test across 20 cases (empty query, malformed reset token, oversized inputs, etc.). PASS = no 500.
<code>test_actual_data.py</code>	Sanity check: counts a known China-IMPORT slice.
<code>test_aggregation_simple.py</code>	Step-by-step aggregation reproducer for the Brazil 0-results bug.
<code>test_brazil.py</code>	One-line: parses <i>“i kinda wanna buy sugar from brazil but only if it’s cheap”</i> .
<code>test_country_fixes.py</code>	Verifies indecisive language doesn’t fabricate country matches.
<code>test_entitlement_internal.py</code>	Two-user entitlement check (locked vs unlocked HS codes).
<code>test_nlu_product.py</code>	GLiNER product-extraction sanity (3 phrasings, expects <code>product=sugar</code>).
<code>test_rapidfuzz.py</code>	Standalone (no Django) RapidFuzz test against 27 standard countries.
<code>test_script.py</code>	Probe: <code>Transaction.objects.filter(seller__icontains='Tereos').first()</code> .
<code>test_security_internal.py</code>	DRF <code>APIClient</code> hits 6 protected endpoints, asserts 401/403 for anonymous.
<code>verify_bug1.py</code>	Verifies prepositions no longer leak into <code>product_keyword</code> post-fix.

Table 7.5: Diagnostic scripts in `backend/` root.

7.8 Cross-references

The data layer interacts with virtually every other chapter. If you edit a model field, you must:

1. Update the model in `backend/<app>/models.py` (Chapter 4).
2. Run `python manage.py makemigrations` and `migrate`.
3. If the field affects search aggregation, update `aggregation.py` (Chapter 5).
4. Update the React component that renders it (Chapter 6).
5. Add a fixture or update `schema_dump.json` if you keep a schema-of-record (Chapter 7).

Chapter 8

Deployment and Operations

This chapter covers everything outside writing application code: containers, the entry-point script, the three setup scripts, the env files, the CI workflow, and how to expose your local server publicly via ngrok.

8.1 Dockerfiles

8.1.1 backend/Dockerfile

```
1 FROM python:3.10-slim
2 ENV PYTHONDONTWRITEBYTECODE=1
3 ENV PYTHONUNBUFFERED=1
4
5 WORKDIR /app
6 RUN apt-get update && apt-get install -y \
7     gcc netcat-openbsd libpq-dev \
8     && rm -rf /var/lib/apt/lists/*
9
10 COPY requirements.txt /app/
11 RUN pip install --upgrade pip && pip install -r requirements.txt
12
13 COPY . /app/
14 COPY ./entrypoint.sh /app/entrypoint.sh
15 RUN chmod +x /app/entrypoint.sh
16
17 ENTRYPOINT ["/app/entrypoint.sh"]
```

Listing 8.1: The backend Dockerfile (657 bytes)

Notes:

- Python 3.10 base; the host setup uses 3.12, so be aware of the small version skew if you containerise locally.
- `netcat-openbsd` is installed because `entrypoint.sh` uses `nc -z` to wait for Postgres.
- No `CMD` — the actual server command must be passed via `docker compose` or the `run` command.

8.1.2 frontend/Dockerfile

```
1 FROM node:18-alpine
2 WORKDIR /app
```

```

3 ENV PATH /app/node_modules/.bin:$PATH
4 COPY package.json /app/package.json
5 COPY package-lock.json /app/package-lock.json
6 RUN npm install --silent
7 COPY . /app
8 EXPOSE 3000
9 CMD ["npm", "start"]

```

Listing 8.2: The frontend Dockerfile (381 bytes)

8.2 The entrypoint

`backend/entrypoint.sh` (321 bytes) is the script the backend container runs at boot:

```

1 #!/bin/sh
2 if [ "$DATABASE" = "postgres" ]; then
3     echo "Waiting for postgres..."
4     while ! nc -z $SQL_HOST $SQL_PORT; do sleep 0.1; done
5     echo "PostgreSQL started"
6 fi
7 echo "Applying database migrations..."
8 python manage.py migrate
9 echo "Starting server..."
10 exec "$@"

```

Listing 8.3: entrypoint.sh

The pattern: wait for the DB to come up, run migrations, then `exec` whatever was passed (typically `gunicorn` or `python manage.py runserver`).

8.3 The two docker-compose files

There are *two*, with very different purposes.

8.3.1 Root `docker-compose.yml` — OpenSearch

Brings up an OpenSearch single-node and the OpenSearch Dashboards UI. Used only if you want to enable the OpenSearch hybrid index path (which is currently gated off in the live runtime; see Chapter 5).

```

1 docker compose up -d
2 # OpenSearch:          http://localhost:9200
3 # OpenSearch UI:       http://localhost:5601

```

Listing 8.4: Bringing up OpenSearch

8.3.2 `backend/docker-compose.yml` — Redis

This is the one the setup scripts launch. Brings up a single `redis/redis-stack` container named `zarailink-redis`.

```

1 cd backend && docker compose up -d
2 # Redis:          redis://localhost:6379
3 # RedisInsight UI: http://localhost:8001

```

Listing 8.5: Bringing up Redis

Beginner Note

The compose file declares `REDIS_ARGS="-requirepass password123"`, but the Django side connects without a password and that just works on Redis-Stack's default config. If you ever flip on strict auth, update `settings.py:131`.

8.4 The three setup scripts (one per OS)

All three scripts have the same logical structure:

1. **Pre-flight** — verify `python3`, `node`, `npm`, `docker`, `psql` are installed and Postgres service is running.
2. **Backend** — create venv if missing or broken, `pip install -r requirements.txt`, copy `.env.example` to `.env`, generate a fresh `SECRET_KEY`.
3. **Database** — `createdb zarailink`, run `migrate`, run `setup_company_roles`, load sample companies if empty, ingest trade Excel if present, build GNN graphs and embeddings.
4. **Docker (Redis)** — detect `docker compose` vs legacy `docker-compose` and start the Redis container.
5. **Frontend** — `cd ../frontend` and `npm install -legacy-peer-deps -silent` if `node_modules/` is missing.
6. **Puppeteer** — in `backend/`, `npm install -silent` (uses `backend/package.json` for the Puppeteer dep used by server-side PDF generation).
7. **Banner** — print run instructions and access URLs.

8.4.1 Per-OS specifics

- **Linux** (`setup_linux.sh`, 16,591 bytes, executable). `sudo -u postgres createdb zarailink ; systemctl` for Postgres; GNU `sed -i "..."` (no empty quotes).
- **macOS** (`setup_mac.sh`, 15,209 bytes). `createdb -U postgres zarailink || createdb zarailink` (Homebrew Postgres has no `postgres` superuser by default); BSD `sed -i "` (empty string after `-i`).
- **Windows** (`setup_windows.ps1`, 14,731 bytes). PowerShell port. `\$env:PGPASSWORD = "postgres"; creat`
`SECRET_KEY` generated via inline Python and patched with `Get-Content/Set-Content`.
Uses `docker-compose up -d` (no `docker compose` fallback).

8.5 Environment variables

8.5.1 Backend .env

Cross-checked against `settings.py`:

Variable	Required?	Effect
SECRET_KEY	Yes	Django cryptographic key.
OPENROUTER_API_KEY	No	LLM-backed price fallback.
OPENAI_KEY	No	AI Smart Search + embedding fallback.
EMAIL_HOST_USER	No	Gmail address for outbound mail.
EMAIL_HOST_PASSWORD	No	Gmail App Password (16 chars).
FRONTEND_URL	No	Used in email templates.

Table 8.1: Backend env variables.

8.5.2 Frontend .env

A single variable: `REACT_APP_API_BASE_URL`.

- `frontend/.env`: `http://localhost:8000` (dev default).
- `frontend/.env.local`: same (per-developer override).
- `frontend/.env.production`: `https://whinier-winter-ruttily.ngrok-free.dev` (the team's persistent ngrok demo URL).

Warning

CRA only exposes env vars whose name starts with `REACT_APP_`. Others are silently ignored at build time.

8.6 Ignore files

- `.gitignore` (root) — `pyc`, `venv`, `logs`, `db.sqlite3`, `node_modules/`, IDE files.
- `backend/.gitignore` — `bytecode`, `venv` variants, `*.log`, `db.sqlite3`, `staticfiles/`, `media/`, `.env`.
- `frontend/.gitignore` — `node_modules/`, `coverage/`, `build/`, `.env.*`.
- `.dockerignore` — `.git`, `__pycache__`, `venv/`, `node_modules/`, IDE files.
- `.gitattributes` — declares `*.pt` and `*.safetensors` as Git LFS tracked. So model weight files use LFS.

8.7 Continuous integration

`.github/workflows/ci.yml` runs on push and PR to `main`. Two parallel jobs:

1. **backend-test** (Ubuntu, Python 3.10): `pip install -r requirements.txt \&\& pip install pytest pyte`
2. **frontend-test** (Ubuntu, Node 18 with npm cache via `package-lock.json`): `npm ci \&\& npm test - --wat`

8.8 Running in production (recommended layout)

The Dockerfiles assume you run `gunicorn` (or similar) as the entry-point command. A typical production `docker-compose.prod.yml` would look like:

```

1 services:
2   db:
3     image: postgres:15
4     environment:
5       POSTGRES_DB: zarailink
6       POSTGRES_USER: postgres
7       POSTGRES_PASSWORD: postgres
8     volumes: [pgdata:/var/lib/postgresql/data]
9
10  redis:
11    image: redis/redis-stack:latest
12    ports: ["6379:6379"]
13
14  api:
15    build: ./backend
16    command: gunicorn zarailink.wsgi:application --bind 0.0.0.0:8000 --workers
17    3
18    environment:
19      DATABASE: postgres
20      SQL_HOST: db
21      SQL_PORT: "5432"
22      SECRET_KEY: ${SECRET_KEY}
23    depends_on: [db, redis]
24    ports: ["8000:8000"]
25
26  web:
27    build: ./frontend
28    ports: ["3000:3000"]
29    depends_on: [api]
30
31  volumes:
32    pgdata:

```

Listing 8.6: Production stack sketch

In production you would also:

- Set `DEBUG = False` in settings (currently hardcoded `True`).
- Set `SESSION_COOKIE_SECURE = True` and `CSRF_COOKIE_SECURE = True`.
- Tighten `ALLOWED_HOSTS` to the real domain.
- Move secrets out of `.env` into Docker secrets, AWS Parameter Store, or similar.
- Run `python manage.py collectstatic` and serve `staticfiles/` via nginx.

8.9 Tunnelling with ngrok

The repo ships `backend/ngrok.exe` (Windows) for quickly exposing `localhost:8000`. The settings already whitelist ngrok hosts:

```

ALLOWED_HOSTS = ['localhost', '127.0.0.1', '.ngrok-free.dev', '.ngrok-free.app
']

```

To start a public tunnel:

```
ngrok http 8000    # gives you https://xxx.ngrok-free.app
```

Then update `frontend/.env.production` (or pass the ngrok URL into `REACT_APP_API_BASE_URL` at build time) to point the React build at the same tunnel.

Chapter 9

Testing

The Zarailink project ships several layers of tests — pytest fixtures and unit tests on the backend, Jest + React Testing Library on the frontend, plus a collection of ad-hoc reliability/perf scripts in `backend/` root. This chapter explains how to run all of them and how to write new ones.

9.1 Backend: pytest + pytest-django

9.1.1 Configuration

`backend/pytest.ini` sets:

```
[pytest]
DJANGO_SETTINGS_MODULE = zarailink.settings
python_files            = test_*.py *_test.py
python_classes          = Test*
python_functions        = test_*
addopts                 = --reuse-db -p no:cacheprovider
```

`-reuse-db` keeps the test DB between runs (much faster); `-p no:cacheprovider` disables pytest's cache plugin so stale caches don't bite.

9.1.2 Fixtures (`backend/conftest.py`)

The `conftest` is 392 lines and provides 28 reusable fixtures + 2 helpers. Categories:

- **Users** — `user_factory`, `user`, `authenticated_user`, `staff_user`, `superuser`.
- **Companies** — `sector`, `company`, `verified_company`, `key_contact`, `public_key_contact`.
- **Subscriptions** — `subscription_plan`, `redeem_code`, `token_purchase`.
- **Trade** — `transaction_factory`, `import_transaction`, `export_transaction`.
- **API clients** — `api_client`, `authenticated_client`.

9.1.3 Test files

- `backend/accounts/tests/test_models.py`

- backend/accounts/tests/test_views.py
- backend/companies/tests/test_views.py
- backend/subscriptions/tests/test_views.py
- backend/search/tests/test_scope_consistency.py

9.1.4 Running the suite

```
cd backend && source .venv/bin/activate
pytest                                # full suite
pytest accounts/tests/                # one app
pytest -k "test_login"                # match test name
pytest -x -vv                         # stop at first fail, verbose
pytest --cov=accounts --cov=companies # coverage
```

9.1.5 Writing a new test

```
1 def test_unlock_charges_one_token(authenticated_client,
2                                   authenticated_user,
3                                   key_contact):
4     starting = authenticated_user.token_balance
5     response = authenticated_client.post(
6         f"/api/key-contacts/{key_contact.id}/unlock/")
7     assert response.status_code == 200
8     assert response.data["status"] == "success"
9     authenticated_user.refresh_from_db()
10    assert authenticated_user.token_balance == starting - 1
```

Listing 9.1: Sample test using the fixtures

9.2 Frontend: Jest + React Testing Library

9.2.1 Tests in the repo

- frontend/src/components/Common/__tests__/ — tests for `Pagination`, `EmptyState`, `VerificationBadge`, `WatchlistButton`.
- frontend/src/hooks/__tests__/ — tests for `useDebounce`, `useToast`, `useWatchlist`.
- frontend/src/context/__tests__/ThemeContext.test.js.

9.2.2 Running

```
cd frontend
npm test                                # watch mode (default CRA)
npm test -- --watchAll=false            # CI mode (single run)
npm test -- --coverage                  # with coverage report
```

9.2.3 Sample frontend test

Pattern used by the existing tests:

```

1 import { render, screen } from '@testing-library/react';
2 import userEvent from '@testing-library/user-event';
3 import { Pagination } from '../Pagination';
4
5 test('disables Previous on first page', () => {
6   const onChange = jest.fn();
7   render(<Pagination current={1} total={5} onChange={onChange} />);
8   const prev = screen.getByRole('button', { name: /previous/i });
9   expect(prev).toBeDisabled();
10 });

```

Listing 9.2: Component test sketch

9.3 Ad-hoc reliability/perf scripts

These are not pytest tests — they spawn the dev server as a subprocess and hit it like a real user would. Useful for sanity checks before a release.

- `backend/perf_test.py` — targets: 500 ms autocomplete, 3000 ms anything else. Spawns server, hits 7 endpoints 3× each, prints a PASS/FAIL grid.
- `backend/reliability_test.py` — 20 negative-input cases (empty queries, oversized strings, gibberish, malformed tokens). PASS == status code other than 500.
- `backend/test_security_internal.py` — DRF `APIClient` hits 6 protected endpoints, asserts 401/403 for the anonymous user.

To run any of them:

```

cd backend && source .venv/bin/activate
python perf_test.py
python reliability_test.py

```

9.4 The CI pipeline

`.github/workflows/ci.yml` runs both test suites on push/PR to `main`. See Chapter 8, §CI for the YAML.

9.5 Manual smoke test (the 6 checks)

Outside automated tests, the team uses a quick manual checklist after significant changes:

1. `curl http://localhost:8000/admin/` returns the Django log-in page.
2. Frontend at `http://localhost:3000` renders without console errors.
3. Sign up a new user; you receive the verification email (or set `is_verified=True` in admin).
4. Visit `/dashboard`; charts render.
5. Run a search; results return in < 2 seconds.
6. Click a result; `DealDetail` loads with all four tabs.

Chapter 10

Troubleshooting

A scrapbook of failures the team has seen and how they were fixed. Use `Ctrl-F` on the error string.

10.1 Setup failures

ModuleNotFoundError: No module named 'django'. Cause: virtualenv is not active. Fix: `source backend/.venv/bin/activate`.

FATAL: database "zarailink" does not exist. Run:

```
sudo -u postgres createdb zarailink      # Linux
createdb zarailink                       # macOS
createdb -U postgres zarailink           # Windows
```

FATAL: password authentication failed for user "postgres". The settings hardcode `password=postgres` (`settings.py:105`). Set it:

```
sudo -u postgres psql -c "ALTER USER postgres PASSWORD 'postgres';"
```

Redis connection refused. Either start the container (`cd backend \&\& docker compose up -d`) or ignore it — Django falls back to LocMem.

npm ERR! ERESOLVE unable to resolve dependency tree. Re-run with the legacy peer-deps flag: `npm install --legacy-peer-deps`.

pip install stalls on torch. The PyTorch wheel is ~700 MB. Be patient. If you have a corporate proxy, install from a local mirror instead.

psql: command not found. Add the PostgreSQL `bin/` folder to your `PATH`.

python3 -m venv fails on Ubuntu. Install the `python3-venv` package: `sudo apt install python3-venv`.

Docker permission denied. Add yourself to the `docker` group and *log out + back in*:

```
sudo usermod -aG docker $USER.
```

10.2 Runtime failures

Browser shows CSRF verification failed. Two common causes:

1. Frontend hosted at a different origin than what `CORS_ALLOWED_ORIGINS` or `CSRF_TRUSTED_ORIGINS` permits. Add the origin to both lists in `settings.py:336-379`.
2. Axios was created without `withCredentials: true`, so the session cookie is not being sent. Confirm `services/api.js` is the only HTTP client.

Search returns “no results” for a query that should have data. Check the response payload for `scope_mismatch`. The likely cause: you searched in IMPORT scope but the data lives in EXPORT (or vice-versa). The frontend should render a banner asking to switch.

Search auto-corrects to the wrong product. The PASS 5 RapidFuzz fallback may have picked a near match. To diagnose, run `backend/diagnose_typo_queries.py` which traces every NLU stage.

The LightGBM model file is missing (warning at boot). `backend/search/models/lgbm_ltr.txt` is not committed; train it once:

```
cd backend && source .venv/bin/activate
python -m search.services.train_ltr
```

Without it, ranking degrades to pure heuristic with no error.

The search index pickle is missing. Build it:

```
python manage.py build_search_index
```

The intent model is missing. The `backend/models/zarai_intent_model/` directory is Git-LFS tracked; if you cloned without `git lfs pull`, the file is a text pointer, not the real binary. Run `git lfs install && git lfs pull`, or retrain via `python -m search.services.train_intent`.

Smart Search returns empty. `OPENAI_KEY` is missing or invalid. Add it to `backend/.env` and restart.

ConnectionError when calling OpenRouter. `OPENROUTER_API_KEY` missing. The price-extraction fallback will skip and the engine returns no price filter — not fatal.

User registers but cannot log in. `accounts/views.py:55-57` *should* set `is_active=True`. If it does not, the user sits at `is_active=False`. Check the form save (lines 44–47) hasn’t been re-broken.

Token deduction races (rare). The `KeyContactViewSet.unlock` action is wrapped in `transaction.atomic`, but if you split the deduct + create across two requests you can double-spend. Keep it atomic.

NLU cache stale after a model retrain. The cache key is `nlu_<md5(query)>`. After re-training the SetFit model, flush:

```
python manage.py shell -c "from django.core.cache import cache; cache.clear()"
```

10.3 Frontend issues

useCompanyNotes is not a function. See Chapter 6: `hooks/index.js` re-exports five hooks whose source files do not exist. `components/TradeDirectory/CompanyNotes.js` will throw on import. Either ship the missing hooks or remove the import.

Dark mode appears half-on. The CSS variables for dark mode exist in `src/index.css`, but no code sets the `dark` class on `<html>`. Wire it in `ThemeContext.js`:

```
useEffect(() => {
  document.documentElement.classList.toggle('dark', darkMode);
}, [darkMode]);
```

Forgot/Reset password do nothing. `ForgotPassword.js` and `ResetPassword.js` are mocked. Wire them to `POST /accounts/api/forgot-password/` and the Django reset-confirm view.

PDF export fails on the comparison page. Inspect the browser console; the most common cause is a `jsPDF` undefined when the table is empty. Guard the call with `if (!suppliers.length) return;`.

10.4 Database issues

relation does not exist. You forgot to migrate. Run `python manage.py migrate`.

Migrations conflict on a fresh checkout. If two branches edited the same model, run:

```
python manage.py makemigrations --merge
python manage.py migrate
```

loaddata fails on duplicate content types. The fixture you are loading includes `contenttypes.contenttype` rows that conflict with auto-created ones. Use `load_data_clean.json` (which strips them) instead of `load_data.json`.

10.5 Performance issues

Search is slow on the first call. The NLU warmup is async; the first request can lose the race and trigger model loading. To guarantee warmup before traffic, wait ~5s after the server logs `[ZaraiLink] NLU warmup complete`.

Aggregation queries time out on big data sets. The aggregator does not paginate the SQL — it pulls every matching transaction. For very large hot subcategories, add a `LIMIT` or a date-range filter at the call-site.

Frontend bundle is large. The dashboard page imports a lot eagerly. Check `App.js` and consider making FindSuppliers / FindBuyers / Subscription lazy too.

10.6 When all else fails

1. Re-read the error in the terminal slowly.
2. Check `backend/debug.log` (errors land there).
3. Search this chapter.
4. `git log -oneline - <file>` to see who touched the file last.
5. Open `problems_solved/country_filter_zero_results.md` — the team's only postmortem so far. It documents the *scope-country conflict* and the *preposition-leak* bugs and the fixes.

Appendix A

Glossary

A flat alphabetical list of terms used in this guide.

Aggregator The component that rolls up raw transactions into per-supplier metrics. See `aggregation.py`.

Auditlog Third-party Django app that records every change to registered models. Configured in `companies/models.py`.

BM25 A sparse text-similarity score used by traditional keyword search. The legacy retriever uses it on subcategory names.

Cold-start LTR Training a learn-to-rank model with synthetic / pseudo labels before real user click data is available. See `ltr_dataset_builder.py`.

CRA Create React App, the toolchain the frontend is built on.

CSRF Cross-Site Request Forgery. Django enforces a token; the frontend reads `csrftoken` from `document.cookie` and includes it as `X-CSRFToken` on POSTs that need it.

DRF Django REST Framework. The library that powers the API viewsets, serializers, and routers.

Entitlement A persisted record that a user has paid (in tokens) to access a particular HS code or product. Stored as `subscriptions.UserAccess`.

FAISS Facebook AI Similarity Search — the in-process ANN index used in `vector_store.py` for the legacy semantic retrieval.

Family One of the nine query categories produced by the legacy `QueryInterpreter`. Determines which heuristic weights apply.

GLiNER Zero-shot NER model (`urchade/gliner_base`) used by the NLU engine to extract product, country, quantity, and price spans in one pass.

GraphML An XML graph format. Zarailink ships seven `*.graphml` files for the GNN/Node2Vec pipeline.

HNSW Hierarchical Navigable Small World — the ANN index algorithm used by Redis Stack for vector search.

HS code Harmonised System code — the worldwide product taxonomy (e.g., `1701` = sugar). Zarailink uses HS prefixes to filter and aggregate trade.

HSN Harmonized System Nomenclature — the Indian/Pakistani local extension of HS codes;

- used as a free-text field on `CompanyProduct.hsn_code`.
- Idempotent** A property of an operation: running it multiple times is the same as running it once. The setup scripts and `purchase_access` are idempotent.
- Intent** BUY (the user wants to find sellers) or SELL (find buyers). Detected by the SetFit classifier.
- KeyBERT** A keyword-extraction algorithm built on top of sentence embeddings. Loaded but currently unused in the live NLU engine.
- LambdaRank/LambdaMART** A pairwise learning-to-rank algorithm implemented by LightGBM. Used for the LTR model.
- LightGBM** Microsoft's gradient-boosting library. Hosts the LTR model.
- LTR** Learning to Rank — the discipline of training a model to re-order search results.
- MD5** A fast (insecure for crypto) hash function used to key the NLU cache.
- Migration** A Python file in `migrations/` that records a schema change. Generate with `makemigrations`, apply with `migrate`.
- NDCG@k** Normalized Discounted Cumulative Gain at k — the standard ranking quality metric. The LTR trainer optimises NDCG@5.
- NLU** Natural Language Understanding. The first stage of the search pipeline.
- Node2Vec** A graph-embedding algorithm based on random walks plus skip-gram. Used to embed the GraphML files.
- ORM** Object Relational Mapper. Django's ORM lets you write Python classes (models) that map to SQL tables.
- Pseudo-label** A label produced by a teacher heuristic instead of by a human, used to bootstrap a student model. See `ltr_dataset_builder.py`.
- RapidFuzz** A C++ Levenshtein-distance library. Zarailink uses it for country resolution and product name typo correction.
- Recharts** A declarative React charting library. The active charting library on the frontend.
- Resolver** The Stage-2 component that turns a product keyword into a list of `ProductSubCategory` IDs. See `_resolve_subcategories` in `search_service.py`.
- Scope** The trade direction the user is browsing in: `IMPORT` or `EXPORT`.
- SetFit** A few-shot fine-tuning library that trains a sentence transformer + a sklearn head on a small labelled set. Hosts the intent classifier.
- Singleton** A class with exactly one instance per process. `SearchService` and `FaissVectorStore` are singletons.
- SBERT / Sentence Transformers** A library that produces sentence-level embeddings. Used everywhere semantic similarity appears.
- Token** In Zarailink, a unit of in-app currency. Users earn or buy tokens and spend them on contact unlocks (1 token) or HS-code access (500 / 5,000 tokens). Not to be confused with auth tokens like JWTs.
- Trigram similarity** PostgreSQL's `pg_trgm` extension compares strings by trigrams (3-letter sequences). Used in PASS 3 of the resolver.
- UUID** Universally Unique Identifier — a 128-bit value used for the email-verification token (`accounts/models.py`).
- ViewSet** A DRF class that bundles related actions on a single resource (list, retrieve, custom actions like `unlock`).

Appendix B

Endpoint Reference

A flat reference of every backend HTTP endpoint, grouped by app. The parent path is shown in the section heading.

B.1 /admin/...

Standard Django admin (`settings.py:42` , `django.contrib.admin`). User-facing pages for managing `User`, `Sector`, `Company`, `CompanyProduct`, `KeyContact`, `KeyContactUnlock`, `SubscriptionPlan`, `RedeemCode`, plus the auditlog log entries.

B.2 /accounts/...

M.	URL	Notes
GET/PC	/accounts/register/	HTML signup page
GET/PC	/accounts/login/	alias to <code>/accounts/register/</code>
POST	/accounts/api/signup/	JSON signup; auto-verifies email
POST	/accounts/api/login/	sets session cookie
POST	/accounts/api/logout/	destroys session
GET	/accounts/api/check-auth/	returns user info or <code>\{authenticated:false\}</code>
POST	/accounts/api/forgot-password/	sends reset email
GET	/accounts/api/verify-email/<uuid:token>/	directs to <code>FRONTEND_URL/verify-email/...</code>
POST	/accounts/api/resend-verification/	multipart HTML+text email
POST	/accounts/api/update-profile/	auth required
POST	/accounts/api/change-password/	auth required, refreshes session
GET	/accounts/password_reset_confirm/<uuid:token>/	Django stock view
GET	/accounts/password_reset_complete/	Django stock view

B.3 /api/ (companies app)

M.	URL	Query params / notes
GET	/api/companies/	?search=, ?use_ai=, ?region=, ?sector=, ?type=, ?ro
GET	/api/companies/<pk>/	detail with nested products + key con- tacts
GET	/api/companies/regions/	distinct provinces
GET	/api/companies/countries/	distinct countries; ?role=
GET	/api/companies/hsn_codes/	distinct HSN codes
GET	/api/key-contacts/	list; ?company=
GET	/api/key-contacts/<pk>/	with paywall
POST	/api/key-contacts/<pk>/unlock/	charges 1 token (or 0 if public)
GET	/api/sectors/	all sectors
GET	/api/company-types/	all types
GET	/api/company-roles/	all roles

B.4 /api/subscriptions/

M.	URL	Body / notes
GET	/api/subscriptions/plans/	list (no auth)
POST	/api/subscriptions/redeem/	demo bypass; auth req'd
POST	/api/subscriptions/purchase-access/	\{access_type, hscore, subcat_id?, product_name?\}
GET	/api/subscriptions/profile-data/	entitlements + history + telemetry
POST	/api/subscriptions/generate-codes/	staff only; form-encoded

B.5 /api/search/

M.	URL	Query params / notes
GET	/api/search/	?q= (req); also ?scope=, ?hs_code=, ?subcat_id=, ?variant_name=, ?i Paywall preview.
GET	/api/search/autocomplete/	up to 8 suggestions
GET	/api/search/hs-code-tree/	numeric drill-down
GET	/api/search/hs-summary/	alpha or numeric summary
GET	/api/search/hs-dashboard/	analytics dashboard; ?intent=, ?subcat=

M.	URL	Query params / notes
GET	/api/search/supplier-detail/	auth req'd; ?name=, ?query=
GET	/api/search/compare/	auth req'd; ?suppliers=A,B,C
GET	/api/search/supplier-transactions/	auth req'd; paginated

B.6 Catch-all

Every other URL is served the React SPA via the `re_path(r'\^.*$', TemplateView.as_view(...))` catch-all in `backend/zarailink/urls.py:36`. This is what makes client-side routing possible.

B.7 Auth requirements at a glance

- **Public:** search list, autocomplete, hs-code-tree, hs-summary, hs-dashboard, the company directory list+detail, sectors / types / roles, plans list.
- **Auth required:** profile-data, purchase-access, redeem, unlock, supplier-detail, compare, supplier-transactions, update-profile, change-password.
- **Staff only:** `/api/subscriptions/generate-codes/`, all admin URLs.

Appendix C

Model Reference

A flat list of every Django model and the field it carries that is not self-explanatory. Cross-reference against `backend/schema_dump.json` (the schema-of-record) and `backend/ZARAILINK_DATA_DESIGN.md`.

C.1 accounts

C.1.1 User

(`accounts/models.py:38-105`) Custom user inheriting `AbstractUser`. Adds: `email` (unique, login field), `country`, `bio`, `phone_number`, `job_title`, `email_verified`, `verification_token` (UUID, unique), `token_created_at`, `token_balance`.

C.1.2 AuditLog

(`accounts/audit.py:10-46`) Records sensitive actions: `user`, `action_type` (`contact_unlock|bulk_unlock|passw`

C.2 companies

C.2.1 Sector / CompanyRole / CompanyType

Each is `name + description`. Used as `ForeignKey(SET_NULL)` targets from `Company`.

C.2.2 Company

(`companies/models.py:47-114`) The core directory entity. 24 fields including: `name`, `legal_name`, `description`,

Indexes: `('country', 'sector')` and `('verification_status',)`.

C.2.3 CompanyProduct

`company`, `name`, `description`, `variety`, `value_added`, `hsn_code`.

C.2.4 KeyContact

company, name, designation, phone, whatsapp, email, is_public.

C.2.5 KeyContactUnlock

user, key_contact, unlocked_at. unique_together=['user', 'key_contact'].

C.3 subscriptions

C.3.1 SubscriptionPlan

plan_name, price, currency, tokens_included, description, features (JSON).

C.3.2 UserSubscription

user, plan (PROTECT), status (pending|active|cancelled|expired), start_date, end_date, billing_cycle

C.3.3 TokenPurchase

user, plan (SET_NULL), tokens_purchased, price, currency, payment_provider, payment_reference, purch

C.3.4 RedeemCode

code (unique 12-char), plan, status, redeemed_by, created_at, redeemed_at, expires_at. Methods: generate_code(length=12), redeem(user).

C.3.5 UserAccess

user, access_type (HS_CODE|PRODUCT), hscore, subcat_id, tokens_spent, purchased_at. Unique constraint on (user, hscore, subcat_id).

C.4 trade_data

C.4.1 HsToProductMap

hs_code, product_name, notes, updated_at.

C.4.2 Product / ProductCategory / ProductSubCategory / ProductItem

The four-level product hierarchy. Each level has name + hs_code + ForeignKey(parent) (except Product which is the root).

C.4.3 Transaction

(trade_data/models.py:65-142) The trade ledger. 21 fields: source_file, tx_reference, reporting_date, tra
Seven Meta indexes covering every hot search filter.

C.4.4 CompanyEmbedding

```
company_name (unique), embedding (JSON), cluster_tag, pagerank, degree.
```

C.4.5 ProductEmbedding

```
product_item (unique), embedding (JSON), cluster_tag.
```

C.5 auditlog (third-party)

The `auditlog` app's `LogEntry` model captures every change to registered models (`Company` , `CompanyProduct` , `KeyContact`). Fields include `actor`, `action`, `changes` (JSON), `timestamp`, `content_type`, ob

C.6 Models with no migrations

`accounts.audit.AuditLog` is defined but *not* auto-migrated because Django only auto-discovers `models.py`. If you want this table, create a manual migration.

`trade_data` also has commented-out auditlog registrations for `Product`, `ProductCategory`, `ProductSubCategory` (lines 180–187). Uncomment if you want change tracking on the catalog.

Appendix D

Quality Checklist

This is the checklist used to verify that this guide is complete, accurate, and useful. Each item links to the section that should satisfy it. If you make changes to the codebase, walk this list to decide what part of the guide needs updating.

D.1 Completeness

D.1.1 Project introduction

- ☐ Explains what Zarailink is and who it is for. (§1)
- ☐ Names the four-stage search pipeline at a high level. (§1.3)
- ☐ Lists every Django app and its responsibility. (Table 1.2)
- ☐ Provides a tech-stack table. (Table 1.1)

D.1.2 Setup

- ☐ Gives prerequisite versions for Python, Node, npm, Docker, PostgreSQL. (Table 2.1)
- ☐ Provides one-shot install commands for Linux, macOS, Windows.
- ☐ Walks the Linux setup script line by line.
- ☐ Documents every environment variable.
- ☐ Provides a six-check smoke test.
- ☐ Lists at least eight common setup errors with fixes.

D.1.3 Architecture

- ☐ Provides a layered architecture diagram. (Figure 3.1)
- ☐ Traces a search request end-to-end. (§3.2)
- ☐ Explains the auth flow.
- ☐ Documents the middleware stack and rationale.

D.1.4 Backend

- ☐ Documents every `settings.py` block.
- ☐ Documents every URL prefix and the catch-all.
- ☐ Covers every Django app: `accounts`, `companies`, `subscriptions`, `trade_data`, `search`.

- ☐ Lists every model and its non-trivial fields.
- ☐ Lists every endpoint with method, URL, and behaviour.
- ☐ Documents every management command.
- ☐ Explains the token-paywall service (`services.py`).

D.1.5 Search engine

- ☐ Diagrams the four stages.
- ☐ Explains `ModernNLUEngine` step by step.
- ☐ Lists the 9-family taxonomy.
- ☐ Explains the 5-pass subcategory resolver.
- ☐ Explains the SQL aggregation kernel and the intent/scope matrix.
- ☐ Lists the 8 LTR features.
- ☐ Explains the 70/30 ensemble formula.
- ☐ Explains the LightGBM training loop.
- ☐ Lists the five preset rankers.
- ☐ Lists the known gaps (missing artefacts, disabled OpenSearch path, etc.).

D.1.6 Frontend

- ☐ Lists every dependency and its use.
- ☐ Lists every route with wrapper.
- ☐ Documents the auth and theme contexts.
- ☐ Documents every custom hook.
- ☐ Documents every shared component.
- ☐ Documents every page (Dashboard, Search, Directory, Intelligence, Watchlist, Subscription, Profile).
- ☐ Calls out the four open issues (`useCompanyNotes` missing, dark mode dormant, ForgotPassword mocked, exportUtils stub).

D.1.7 Data

- ☐ Tabulates every Excel and CSV.
- ☐ Tabulates every JSON fixture.
- ☐ Tabulates every GraphML file.
- ☐ Tabulates every ML artefact.
- ☐ Documents every ingestion script.

D.1.8 Deployment

- ☐ Reproduces both Dockerfiles verbatim.
- ☐ Documents the entrypoint script.
- ☐ Documents both docker-compose files.
- ☐ Walks all three setup scripts.
- ☐ Documents every env variable.
- ☐ Mentions Git LFS for model weights.
- ☐ Documents the CI workflow.

D.1.9 Testing

- ☐ Explains pytest config and fixtures.
- ☐ Lists every test file.
- ☐ Explains the frontend test setup.
- ☐ Lists ad-hoc reliability/perf scripts.

D.1.10 Troubleshooting

- ☐ Covers setup errors.
- ☐ Covers runtime errors.
- ☐ Covers frontend issues.
- ☐ Covers DB issues.
- ☐ Covers performance issues.

D.1.11 Reference

- ☐ Glossary covers every domain-specific term.
- ☐ Endpoints reference is exhaustive.
- ☐ Model reference is exhaustive.

D.2 Accuracy

For each chapter the writer must verify:

- ☐ Every quoted line number was checked against the actual file (use `Ctrl-G` in your editor).
- ☐ Every claimed default value was checked against `settings.py` or the model definition.
- ☐ Every URL string was checked against the relevant `urls.py`.
- ☐ Every command-line invocation was tested in a real terminal.
- ☐ Every claim about a feature flag (e.g. `if False and ...`) was verified to still be in the code.
- ☐ Every screenshot, if added, matches the current UI.

D.3 Self-consistency

- ☐ Cross-references resolve (`\ref\{sec:trace\}` produces a real label).
- ☐ Tables in the appendices match the tables in the main chapters (no contradictions).
- ☐ The known-gaps list in Chapter 5 matches the Troubleshooting chapter.

D.4 Build hygiene

- ☐ PDF compiles cleanly with `pdflatex zarailink_guide.tex` (run twice for TOC).
- ☐ No unresolved references warnings.
- ☐ No overfull `hbox` warnings on section headings.
- ☐ All listings render with line numbers.
- ☐ All tables fit the page.

- ☐ PDF metadata is set (`pdftitle`, `pdfauthor`, `pdfkeywords`).

D.5 Pedagogy

- ☐ An absolute beginner can read Chapter 1 + Chapter 2 and have a running install.
- ☐ Every novel concept (NLU, LTR, NDCG, GraphML, etc.) is defined the first time it appears.
- ☐ Every code block is labelled with a caption that names the file or the concept.
- ☐ Every callout box is appropriately styled (Beginner, Warning, Tip, Worked Example, Why).
- ☐ Diagrams accompany every multi-step process.

D.6 Source-of-truth links

- ☐ Backend exhaustive notes: `/tmp/zarailink_backend_notes.md` (2,283 lines).
- ☐ Search engine exhaustive notes: `/tmp/zarailink_search_notes.md` (2,264 lines).
- ☐ Frontend exhaustive notes: `/tmp/zarailink_frontend_notes.md` (1,536 lines).
- ☐ Data and deployment notes: `/tmp/zarailink_data_deploy_notes.md` (~700 lines).
- ☐ Project's own `COMPLETE_SYSTEM_GUIDE.md` (the canonical narrative).
- ☐ Project's own `ZARAILINK_DATA_DESIGN.md` (the schema-of-record).

Appendix E

Further Resources

A curated collection of pointers — inside the repo and on the open web — for every part of the stack.

E.1 Inside the repository

E.1.1 Documentation

- `COMPLETE_SYSTEM_GUIDE.md` — the team’s 2,500-line narrative walkthrough.
- `README.md` — mega-README; sibling of the system guide.
- `BACKEND_DETAILS.md` — elevator pitch of the backend.
- `SDS_DATABASE_ARCHITECTURE.md` — formal SDS for the database.
- `SEARCH_ENGINE_ARCHITECTURE.md` — narrative of the four search modes.
- `HS_CODE_SEARCH_EVALUATION.md` — design doc for the HS-code drill-down feature.
- `ltr_architecture.md` — LTR rationale and feature table.
- `Data_NLU_Pipeline_Details.md` — polished mini-spec of the NLU pipeline.
- `search.md` — five-phase search engine implementation history.
- `SEARCH_MODULE_README.md` — original setup notes for the `search` app.
- `FRONTEND_SEARCH_README.md` — frontend search component sketch.
- `PROPOSED_UPDATES.md` — 4-feature roadmap.
- `SUGGESTED_IMPROVEMENTS.md` — senior-architect roadmap (note the typo).
- `Zarailink SRS Analysis.md` — SRS-derived module descriptions.
- `TEAM_SETUP.md` — branch-aware April-2026 team guide.
- `SETUP.md` — original developer setup guide.
- `problems_solved/country_filter_zero_results.md` — the team’s only postmortem so far.

- `backend/ZARAILINK_DATA_DESIGN.md` — auto-generated schema reference.
- `details.md` — mermaid-diagram summary of the search ecosystem.

E.1.2 Reference assets

- `nlu_pipeline.svg` and `system_architecture.svg` — the two architecture diagrams.
- `backend/zarailink_erd.png` — entity-relationship diagram.
- `zarailink_fyp_poster.html` — A0 print-ready poster.

E.1.3 Schema-of-record

- `backend/schema_dump.json` — 22 model specs with every field.

E.2 Inside this guide build

The guide LaTeX source lives at `docs/guide/zarailink_guide.tex` with chapter sources in `docs/guide/chapters/`.

To rebuild:

```
cd docs/guide
pdflatex -interaction=nonstopmode zarailink_guide.tex
pdflatex -interaction=nonstopmode zarailink_guide.tex # again, for TOC
```

The first pass compiles content; the second pass resolves cross-references (table of contents, figure list, page numbers).

Notes used as input. The four exhaustive notes files generated during the writing of this guide are at: `/tmp/zarailink_backend_notes.md`, `/tmp/zarailink_search_notes.md`, `/tmp/zarailink_frontend_notes.md`, `/tmp/zarailink_data_deploy_notes.md`. They are not part of the repo but were the sources for almost every specific line-number citation in this PDF.

E.3 Open-web references

E.3.1 Frameworks and libraries

- Django: <https://docs.djangoproject.com/en/4.2/>
- Django REST Framework: <https://www.django-rest-framework.org/>
- React: <https://react.dev/>
- Tailwind CSS: <https://tailwindcss.com/>
- Recharts: <https://recharts.org/>
- Framer Motion: <https://www.framer.com/motion/>
- Axios: <https://axios-http.com/>
- React Router: <https://reactrouter.com/>

E.3.2 ML libraries

- LightGBM: <https://lightgbm.readthedocs.io/>
- SetFit: <https://github.com/huggingface/setfit>
- GLiNER: <https://github.com/urchade/GLiNER>
- Sentence Transformers: <https://www.sbert.net/>
- RapidFuzz: <https://github.com/maxbachmann/RapidFuzz>
- KeyBERT: <https://github.com/MaartenGr/KeyBERT>
- FAISS: <https://github.com/facebookresearch/faiss>
- NetworkX: <https://networkx.org/>
- Node2Vec: <https://snap.stanford.edu/node2vec/>

E.3.3 Infrastructure

- PostgreSQL: <https://www.postgresql.org/docs/>
- PostgreSQL pg_trgm: <https://www.postgresql.org/docs/current/pgtrgm.html>
- Redis Stack: <https://redis.io/docs/about/about-stack/>
- OpenSearch: <https://opensearch.org/docs/>
- Docker Compose: <https://docs.docker.com/compose/>
- ngrok: <https://ngrok.com/docs>

E.3.4 Concepts

- Learning to Rank (NDCG, LambdaRank): https://en.wikipedia.org/wiki/Learning_to_rank
- HS Codes: <https://www.wcoomd.org/en/topics/nomenclature.aspx>
- Pseudo-labelling: see Lee, Dong-Hyun (2013), *Pseudo-Label*.
- NDCG@k: see Wang *et al.*, *A Theoretical Analysis of NDCG Type Ranking Measures*, COLT 2013.

E.4 Where to ask for help

- Internal team channel (see your team Slack/Discord).
- GitHub issues on the project repo (or the upstream forks).
- Stack Overflow tags: `[django]`, `[django-rest-framework]`, `[reactjs]`, `[lightgbm]`.
- For ML-specific questions: HuggingFace forums (<https://discuss.huggingface.co/>).

E.5 Versioning of this guide

This document tracks the codebase as committed on the `umar_branch2` branch as of compilation. When the code evolves significantly, update the relevant chapter and bump the preface's source-of-truth note.